

Univerzitet u Beogradu
Fakultet organizacionih nauka
Katedra za elektronsko poslovanje

Veb aplikacija za uvoznike

Rbr	Ime	Prezime	Broj indeksa
1.	Luka	Grčić	2022/0252
2.	Stefan	Samardžija	2022/0433
3.	Uroš	Puača	2022/0569
Mentor		Milica M. Simić	
GitHub link		https://github.com/elab-development/internet-tehnologije-2025-aplikacijazauvoznike_2022_0252	

1. Korisnički zahtev

1.1. Verbalni opis za izabranu temu

Osnovna funkcija aplikacije je da olakša posao firmama koje se bave uvozom robe. Kroz jednostavan i pregledan prikaz ponude uvoznicima se omogućava brzo i jednostavno naručivanje robe od različitih dobavljača. Tradicionalni način pronalaženja dobavljača, kao i samo kontaktiranje, često je neefikasan i oduzima previše vremena. Upravo taj problem ova aplikacija rešava, centralizujući ponudu i omogućavajući firmama da sve potrebne informacije pronađu na jednom mestu, uz značajnu uštedu vremena i troškova.

Cilj aplikacije je automatizacija procesa naručivanja robe i kreiranja kontejnera za uvoz. Aplikacija omogućava pregled svih ključnih informacija, kao što su cena, naziv, slika i šifra proizvoda. Pored toga, sistem vodi računa o svim ograničenjima vezanim za uvoz, uključujući dimenzije, težinu i troškove kontejnera. Aplikacija takođe optimizuje raspored proizvoda u kontejnerima, sprečava greške u porudžbinama i obaveštava korisnike o potencijalnim prekoračenjima prostora ili troškova. Na taj način značajno ubrzava ceo proces uvoza i doprinosi efikasnijem poslovanju firmi koje se bave trgovinom.

Ciljna grupa aplikacije obuhvata sve dobavljače i uvoznike. Dobavljači putem sistema mogu da unose, ažuriraju i uređuju svoje ponude, čime obezbeđuju uvoznicima uvek tačne i aktuelne informacije o proizvodima. Uvoznici, sa druge strane, imaju mogućnost detaljnog pregleda kompletne ponude, filtriranja proizvoda prema svojim potrebama i izboru opcije koja najbolje odgovara njihovom poslovanju. Nakon odabira proizvoda, korisnici mogu jednostavno kreirati kontejnere u skladu sa svim zadatim ograničenjima, kao što su dimenzije, težina i ukupni troškovi. Na ovaj način aplikacija efikasno povezuje dobavljače i uvoznike, ubrzava proces poslovanja i omogućava jednostavnije, pouzdanije i transparentnije upravljanje uvozom.

1.2. Funkcionalni zahtevi

1. Upravljanje korisnicima

- Sistem omogućava registraciju novih korisnika.
- Sistem omogućava prijavu registrovanih korisnika u aplikaciju.
- Sistem omogućava izmenu podataka postojećeg korisnika

2. Upravljanje saradnjom (administrator)

- Sistem omogućava kreiranje saradnje između uvoznika i dobavljača.
- Sistem omogućava pregled postojećih saradnji.
- Sistem omogućava brisanje postojeće saradnje.

3. Upravljanje ponudama (funkcionalnosti dobavljača)

- Dobavljač može da unese novu ponudu.
- Dobavljač može da obriše postojeću ponudu.
- Dobavljač može da pregleda svoje ponude.
- Dobavljač može da definiše nove vrste proizvoda.
- Dobavljač može da obriše postojeću vrstu proizvoda.

4. Pregled i pretraga ponuda (funkcionalnosti uvoznika)

- Uvoznik može da pregleda dostupne ponude proizvoda dobavljača sa kojima ima aktivnu saradnju.
- Uvoznik može da poredi više ponuda proizvoda prema definisanim kriterijumima.

5. Kreiranje i upravljanje kontejnerima

- Uvoznik može da kreira novi kontejner.
- Uvoznik može da dodaje stavke (proizvode) u kontejner.
- Uvoznik može da uklanja stavke iz kontejnera.
- Uvoznik može da finalizuje kontejner.
- Sistem proverava ograničenja kapaciteta (težina, zapremina, količina).

6. Bezbednost i integritet podataka

- Sistem obezbeđuje autentifikaciju i autorizaciju korisnika.

1.3. Nefunkcionalni zahtevi

1. Performanse

- Sistem omogućava učitavanje stranica i podataka u roku kraćem od 2 sekunde pri normalnom opterećenju.
- Pretraga ponuda se izvršava u roku manjem od 1 sekunde.

2. Bezbednost

- Svi korisnički nalozi su zaštićeni lozinkom koja je heširana i čuvana u sigurnom formatu.
- Pristup funkcijama je kontrolisan na osnovu korisničkih uloga (dobavljač, uvoznik, administrator).

3. Pouzdanost

- Ciljna dostupnost aplikacije je najmanje 99% na mesečnom nivou

4. Skalabilnost i održavanje

- Sistem je projektovan tako da omogućava lako proširenje broja korisnika i količine podataka.
- Baza podataka omogućava rad sa velikim brojem ponuda uz stabilno vreme odziva.
- Izvorni kod aplikacije je organizovan i dokumentovan tako da omogući jednostavno održavanje i nadogradnju sistema.

5. Upotrebljivost (UX/UI)

- Aplikacija ima jednostavan i intuitivan korisnički interfejs.
- Sistem je prilagođen i korisnicima bez tehničkog znanja.

6. Portabilnost

- Aplikacija radi u modernim veb pregledačima (Chrome, Edge, Firefox)

1.4. Uloge nad sistemom i permisije

ULOGA	Dozvoljene funkcije	Ograničenja	Prava pristupa podacima
Admin	Pregled korisnika Izmena korisnika, Kreiranje i brisanje saradnji	Ne može upravljati ponudama, Ne može upravljati vrstama proizvoda, Ne može upravljati kontejnerima	Pristup ostalim korisnicima i saradnjama
Uvoznik	Kreiranje i upravljanje kontejnerima, Dodavanje ponuda u kontejner, Pregled ponuda samo od dobavljača sa kojima sarađuje, Poređenje ponuda	Vidi samo ponude od dobavljača sa kojima sarađuje, Ograničenja kontejnera (dimenzije), Ne može brisati finalizovane kontejnere i stavke u njima	Sopstveni kontejneri, Ponude dobavljača sa kojima sarađuje
Dobavljač	Kreiranje i upravljanje ponudama, Kreiranje i upravljanje vrstama proizvoda	Ne vidi ponude drugih dobavljača, Ne može upravljati kontejnerima	Sopstvene ponude proizvoda, Vrste proizvoda

2. Opis sistema

2.1. Opis slučajeva korišćenja (Use Case opis)

SK1 - Prijavi korisnika

SK2 - Registruj korisnika

SK3 - Izmeni korisnika

SK4 - Kreiraj saradnju

SK5 - Obriši saradnju

SK6 - Kreiraj ponudu proizvoda

SK7 - Obriši ponudu proizvoda

SK8 - Kreiraj vrstu proizvoda

SK9 - Obriši vrstu proizvoda

SK10 - Kreiraj kontejner

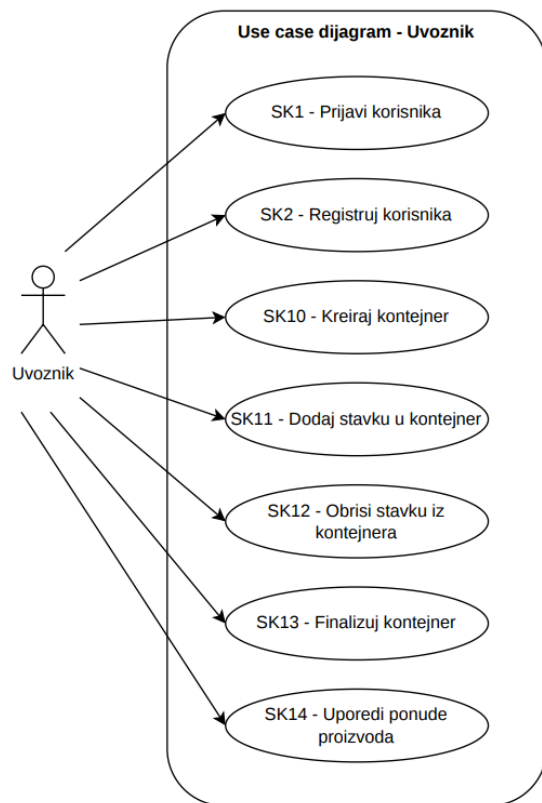
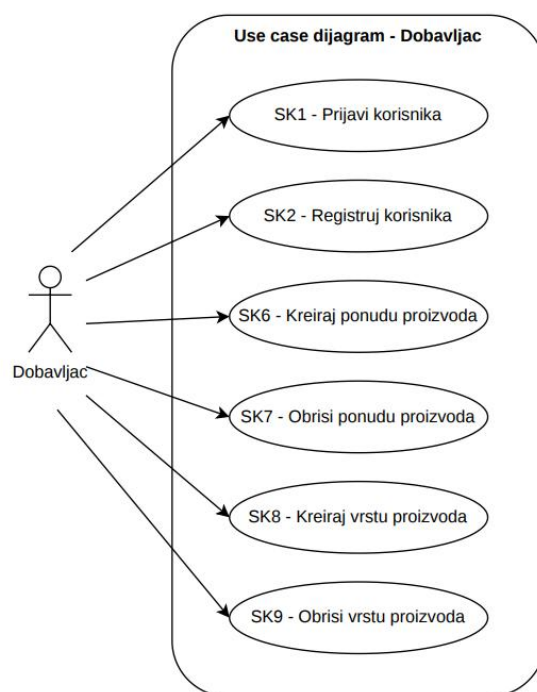
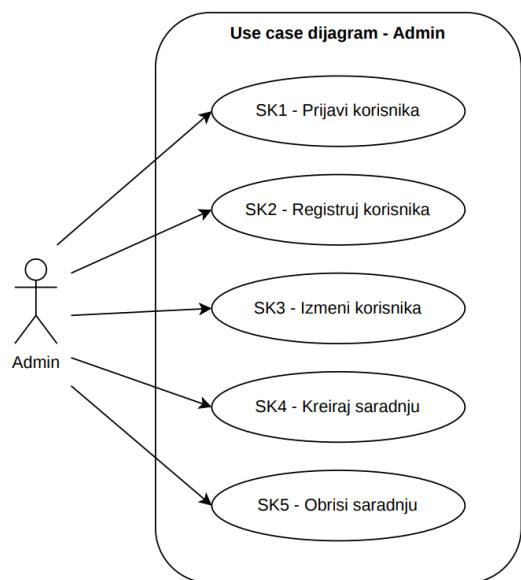
SK11 - Dodaj stavku u kontejner

SK12 - Obriši stavku iz kontejnera

SK13 - Finalizuj kontejner

SK14 - Uporedi ponude proizvoda

Dijagram slučajeve korišćenja za svaki tip korisnika:



SK1: Prijavi korisnika

Glavne uloge (akteri): Korisnik

Kratak opis: Korisnik se prijavljuje na sistem unosom emaila i šifre.

Preduslovi:

- Korisnik nije prijavljen
- Korisnik ima kreiran nalog
- Sistem prijave je dostupan

Glavni tok:

1. Korisnik otvara formu za prijavu. (APUSO)
2. Korisnik unosi email i šifru. (APUSO)
3. Sistem šalje zahtev API-ju. (SO)
4. API pronalazi korisnika u bazi po emailu. (SO)
5. API proverava da li se uneta šifra poklapa sa šifrom u bazi. (SO)
6. Sistem generiše autentifikacioni token i postavlja kolačić. (SO)
7. Sistem preusmerava korisnika na Dashboard. (IA)

Alternativni tokovi:

4.1: Email ili šifra nisu ispravni — sistem prikazuje poruku: *"Invalid email or password."* (IA)

5.1: Uneta šifra se ne poklapa sa šifrom u bazi — sistem prikazuje poruku: *"Invalid email or password."* (IA)

6.1: Greška pri komunikaciji sa serverom — sistem prikazuje poruku: *"Network error. Try again."* (IA)

7.1: Ukoliko API vrati grešku bez poruke — sistem prikazuje poruku: *"Login failed."* (IA)

Postuslovi:

- Korisnik je prijavljen i preusmeren na Dashboard, ili je obavešten o neuspehu prijave.

SK2: Registruj korisnika

Glavne uloge (akteri): Korisnik

Kratak opis: Korisnik kreira nalog unosom podataka o kompaniji i kredencijalima.

Preduslovi:

- Korisnik nije prijavljen
- Sistem registracije je dostupan

Glavni tok:

1. Korisnik otvara formu za registraciju. (APUSO)
2. Korisnik unosi podatke (naziv kompanije, država, adresa, uloga, email, šifra i potvrda šifre). (APUSO)
3. Sistem validira unete podatke. (SO)
4. Sistem šalje zahtev API-ju. (SO)
5. API proverava jedinstvenost emaila i kreira nalog. (SO)
6. Nakon uspešne registracije sistem automatski prijavljuje korisnika i preusmerava ga na Dashboard. (IA)

Alternativni tokovi:

3.1: Naziv kompanije ima manje od 3 karaktera — sistem prikazuje poruku: *"The company name must have at least 3 characters."* (IA)

3.2: Adresa ima manje od 5 karaktera — sistem prikazuje poruku: *"The address must have at least 5 characters."* (IA)

3.3: Šifra ima manje od 6 karaktera — sistem prikazuje poruku: *"Password must have at least 6 characters."* (IA)

3.4: Šifra i potvrda šifre nisu iste — sistem prikazuje poruku: *"The passwords do not match."* (IA)

5.1: Email već postoji u sistemu — sistem prikazuje poruku: *"Email already exists."* (IA)

5.2: Greška pri komunikaciji sa serverom — sistem prikazuje poruku: *"Greška u mreži. Pokušaj ponovo."* (IA)

Postuslovi: Nalog je kreiran i korisnik je automatski prijavljen i preusmeren na Dashboard, ili je obavešten o neuspehu.

SK3: Izmeni korisnika

Glavne uloge (akteri): Admin

Kratak opis: Admin pregledava listu korisnika i može da ažurira podatke bilo kog korisnika.

Preduslovi:

- Korisnik je prijavljen sa ADMIN ulogom
- Sistem je dostupan

Glavni tok:

1. Admin otvara stranicu za upravljanje korisnicima. (APUSO)
2. Sistem proverava da li je korisnik ADMIN. (SO)
3. Sistem učitava i prikazuje listu svih korisnika. (SO)
4. Admin opciono filtrira korisnike po ulozi (ALL, IMPORTER, SUPPLIER). (APUSO)
5. Admin klikne dugme **"Update"** pored željenog korisnika. (APUSO)
6. Sistem otvara modal za izmenu podataka korisnika. (IA)
7. Admin menja podatke i potvrđuje izmenu. (APUSO)
8. Sistem šalje zahtev API-ju sa ažuriranim podacima. (SO)
9. API čuva izmene u bazi i vraća ažurirane podatke. (SO)
10. Sistem ažurira prikaz korisnika u tabeli bez ponovnog učitavanja stranice. (IA)

Alternativni tokovi:

- 2.1: Ukoliko korisnik nije ADMIN — sistem ga preusmerava na Dashboard. (IA)
- 2.2: Ukoliko dođe do greške pri proveru uloge — sistem preusmerava na Dashboard. (IA)
- 3.1: Ukoliko nema korisnika za izabrani filter — sistem prikazuje poruku: *"No users found for the selected filter."* (IA)
- 9.1: Ukoliko ažuriranje ne uspe — sistem prikazuje poruku: *"Update failed."* (IA)

Postuslovi:

- Podaci korisnika su uspešno ažurirani u bazi, ili je admin obavešten o neuspehu.

SK4: Kreiraj saradnju

Glavne uloge (akteri): Admin

Kratak opis: Admin kreira novu saradnju između uvoznika i dobavljača.

Preduslovi:

- Korisnik je prijavljen sa ADMIN ulogom
- U sistemu postoji bar jedan IMPORTER i jedan SUPPLIER

Glavni tok:

1. Admin otvara stranicu za upravljanje saradnjama. (APUSO)
2. Admin klikne dugme **" + New Collaboration "**. (APUSO)
3. Sistem otvara modal za kreiranje saradnje. (IA)
4. Admin bira uvoznika i dobavljača iz padajućih menija. (APUSO)
5. Admin potvrđuje kreiranje klikom na **"Create"**. (APUSO)
6. Sistem šalje zahtev API-ju. (SO)
7. API proverava da li izabrani korisnici postoje i imaju ispravne uloge. (SO)
8. API kreira saradnju u bazi. (SO)
9. Sistem prikazuje novu saradnju u tabeli. (IA)

Alternativni tokovi:

4.1: Ukoliko uvoznik i dobavljač nisu izabrani — sistem prikazuje poruku: *"Importer and supplier are required."* (IA)

4.2: Ukoliko je izabran isti korisnik za oba — sistem prikazuje poruku: *"Importer and supplier cannot be the same user."* (IA)

7.1: Ukoliko izabrani korisnik nije IMPORTER — sistem prikazuje poruku: *"Selected user is not an importer."* (IA)

7.2: Ukoliko izabrani korisnik nije SUPPLIER — sistem prikazuje poruku: *"Selected user is not a supplier."* (IA)

8.1: Ukoliko saradnja već postoji — sistem prikazuje poruku: *"Collaboration between selected importer and supplier already exists."* (IA)

8.2: Ukoliko dođe do greške u bazi — sistem prikazuje poruku: *"Internal server error."* (IA)

Postuslovi: Saradnja je uspešno kreirana, ili je admin obavešten o neuspehu.

SK5: Obrisi saradnju

Glavne uloge (akteri): Admin

Kratak opis: Admin briše postojeću saradnju između uvoznika i dobavljača.

Preduslovi:

- Korisnik je prijavljen sa ADMIN ulogom
- U sistemu postoji bar jedna saradnja

Glavni tok:

1. Admin otvara stranicu za upravljanje saradnjama. (APUSO)
2. Sistem učitava i prikazuje listu svih saradnji. (SO)
3. Admin klikne dugme **"Delete"** pored željene saradnje. (APUSO)
4. Sistem prikazuje dijalog za potvrdu: *"Are you sure you want to delete this collaboration?"* (IA)
5. Admin potvrđuje brisanje. (APUSO)
6. Sistem šalje zahtev API-ju za brisanje. (SO)
7. API briše saradnju iz baze. (SO)
8. Sistem uklanja saradnju iz tabele bez ponovnog učitavanja stranice. (IA)

Alternativni tokovi:

- 2.1: Ukoliko nema saradnji — sistem prikazuje poruku: *"No collaborations found."* (IA)
- 2.2: Ukoliko dođe do greške pri učitavanju — sistem prikazuje poruku o grešci. (IA)
- 4.1: Ukoliko admin odustane od brisanja — sistem zatvara dijalog bez brisanja. (IA)
- 7.1: Ukoliko brisanje ne uspe — sistem vraća listu u prethodno stanje i prikazuje poruku o grešci. (IA)

Postuslovi:

- Saradnja je uspešno obrisana, ili je admin obavešten o neuspehu.

SK6: Kreiraj ponudu proizvoda

Glavne uloge (akteri): Dobavljač (Supplier)

Kratak opis: Dobavljač kreira novu ponudu proizvoda unosom svih potrebnih podataka.

Preduslovi:

- Korisnik je prijavljen sa SUPPLIER ulogom
- Postoji bar jedna kategorija proizvoda u sistemu

Glavni tok:

1. Dobavljač otvara stranicu sa svojim ponudama. (APUSO)
2. Dobavljač klikne dugme **" + New Offer "**. (APUSO)
3. Sistem otvara modal za kreiranje ponude. (IA)
4. Dobavljač unosi podatke — kategoriju, šifru, naziv, opis, URL slike, cenu i dimenzije. (APUSO)
5. Dobavljač potvrđuje kreiranje klikom na **"Save"**. (APUSO)
6. Sistem šalje zahtev API-ju. (SO)
7. API proverava token i ulogu korisnika. (SO)
8. API kreira ponudu u bazi i vraća ID nove ponude. (SO)
9. Sistem osvežava listu ponuda i prikazuje novu ponudu. (IA)

Alternativni tokovi:

7.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)

7.2: Ukoliko korisnik nije SUPPLIER — sistem vraća grešku: *"Forbidden."* (IA)

8.1: Ukoliko neko obavezno polje nije popunjeno — sistem vraća grešku: *"Missing required fields."* (IA)

8.2: Ukoliko dođe do greške u bazi — sistem vraća grešku: *"Internal server error."* (IA)

Postuslovi:

- Ponuda je uspešno kreirana i prikazana u listi, ili je dobavljač obavešten o neuspehu.

SK7: Obrisi ponudu proizvoda

Glavne uloge (akteri): Dobavljač (Supplier)

Kratak opis: Dobavljač briše postojeću ponudu proizvoda.

Preduslovi:

- Korisnik je prijavljen sa SUPPLIER ulogom
- Postoji bar jedna ponuda u sistemu

Glavni tok:

1. Dobavljač otvara stranicu sa svojim ponudama. (APUSO)
2. Sistem učitava i prikazuje listu svih ponuda. (SO)
3. Dobavljač klikne dugme **"Delete"** pored željene ponude. (APUSO)
4. Sistem prikazuje dijalog za potvrdu: *"Are you sure you want to delete this offer?"* (IA)
5. Dobavljač potvrđuje brisanje. (APUSO)
6. Sistem šalje zahtev API-ju za brisanje. (SO)
7. API proverava token i ulogu korisnika. (SO)
8. API briše ponudu iz baze. (SO)
9. Sistem uklanja ponudu iz liste bez ponovnog učitavanja stranice. (IA)

Alternativni tokovi:

- 4.1: Ukoliko dobavljač odustane od brisanja — sistem zatvara dijalog bez brisanja. (IA)
- 7.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)
- 7.2: Ukoliko korisnik nije SUPPLIER — sistem vraća grešku: *"Forbidden."* (IA)
- 8.1: Ukoliko ponuda ne postoji ili ne pripada dobavljaču — sistem vraća grešku: *"Not found."* (IA)

Postuslovi:

- Ponuda je uspešno obrisana iz liste, ili je dobavljač obavešten o neuspehu.

SK8: Kreiraj vrstu proizvoda

Glavne uloge (akteri): Dobavljač (Supplier)

Kratak opis: Dobavljač kreira novu vrstu (kategoriju) proizvoda.

Preduslovi:

- Korisnik je prijavljen sa SUPPLIER ulogom
- Sistem je dostupan

Glavni tok:

1. Dobavljač otvara stranicu sa kategorijama proizvoda. (APUSO)
2. Dobavljač klikne dugme **" + New Category "**. (APUSO)
3. Sistem otvara modal za kreiranje kategorije. (IA)
4. Dobavljač unosi naziv kategorije. (APUSO)
5. Dobavljač potvrđuje kreiranje klikom na **"Save"**. (APUSO)
6. Sistem šalje zahtev API-ju. (SO)
7. API proverava token i ulogu korisnika. (SO)
8. API kreira kategoriju u bazi i vraća kreiranu kategoriju. (SO)
9. Sistem osvežava listu kategorija i prikazuje novu kategoriju. (IA)

Alternativni tokovi:

- 7.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)
- 7.2: Ukoliko korisnik nije SUPPLIER — sistem vraća grešku: *"Forbidden."* (IA)
- 8.1: Ukoliko naziv nije unet — sistem vraća grešku: *"Name is required."* (IA)
- 8.2: Ukoliko kategorija sa istim nazivom već postoji — sistem vraća grešku: *"Category already exists."* (IA)
- 8.3: Ukoliko dođe do greške u bazi — sistem vraća grešku: *"Internal server error."* (IA)

Postuslovi:

- Kategorija je uspešno kreirana i prikazana u listi, ili je dobavljač obavešten o neuspehu.

SK9: Obrisi vrstu proizvoda

Glavne uloge (akteri): Dobavljač (Supplier)

Kratak opis: Dobavljač briše postojeću vrstu (kategoriju) proizvoda.

Preduslovi:

- Korisnik je prijavljen sa SUPPLIER ulogom
- Postoji bar jedna kategorija u sistemu

Glavni tok:

1. Dobavljač otvara stranicu sa kategorijama proizvoda. (APUSO)
2. Sistem učitava i prikazuje listu svih kategorija. (SO)
3. Dobavljač klikne dugme **"Delete"** pored željene kategorije. (APUSO)
4. Sistem prikazuje dijalog za potvrdu: *"Are you sure you want to delete this category?"* (IA)
5. Dobavljač potvrđuje brisanje. (APUSO)
6. Sistem šalje zahtev API-ju za brisanje. (SO)
7. API proverava token i ulogu korisnika. (SO)
8. API briše kategoriju iz baze. (SO)
9. Sistem uklanja kategoriju iz liste bez ponovnog učitavanja stranice. (IA)

Alternativni tokovi:

- 4.1: Ukoliko dobavljač odustane od brisanja — sistem zatvara dijalog bez brisanja. (IA)
- 7.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)
- 7.2: Ukoliko korisnik nije SUPPLIER — sistem vraća grešku: *"Forbidden."* (IA)
- 8.1: Ukoliko kategorija ne postoji — sistem vraća grešku: *"Not found."* (IA)

Postuslovi:

- Kategorija je uspešno obrisana iz liste, ili je dobavljač obavešten o neuspehu.

SK10: Kreiraj kontejner

Glavne uloge (akteri): Uvoznik (Importer)

Kratak opis: Uvoznik kreira novi kontejner unosom dimenzija i opcionalnog naziva.

Preduslovi:

- Korisnik je prijavljen sa IMPORTER ulogom
- Sistem je dostupan

Glavni tok:

1. Uvoznik otvara stranicu sa kontejnerima. (APUSO)
2. Uvoznik klikne dugme **" + New Container "**. (APUSO)
3. Sistem otvara modal za kreiranje kontejnera. (IA)
4. Uvoznik unosi opcioni naziv i dimenzije kontejnera. (APUSO)
5. Uvoznik potvrđuje kreiranje klikom na **"Create"**. (APUSO)
6. Sistem šalje zahtev API-ju. (SO)
7. API proverava token i ulogu korisnika. (SO)
8. API proverava ispravnost dimenzija i kreira kontejner u bazi. (SO)
9. Sistem osvežava listu kontejnera i prikazuje novi kontejner. (IA)

Alternativni tokovi:

7.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)

7.2: Ukoliko korisnik nije IMPORTER — sistem vraća grešku: *"Forbidden."* (IA)

8.1: Ukoliko dimenzije nisu unete — sistem vraća grešku: *"Invalid dimensions."* (IA) 8.2: Ukoliko širina nije između 2 i 3 metra — sistem vraća grešku: *"Width must be between 2.00 and 3.00 meters."* (IA)

8.3: Ukoliko visina nije između 2 i 3 metra — sistem vraća grešku: *"Height must be between 2.00 and 3.00 meters."* (IA)

8.4: Ukoliko dubina nije između 5 i 13 metara — sistem vraća grešku: *"Depth must be between 5.00 and 13.00 meters."* (IA)

Postuslovi:

- Kontejner je uspešno kreiran sa statusom DRAFT, ili je uvoznik obavešten o neuspehu.

SK11: Dodaj stavku u kontejner

Glavne uloge (akteri): Uvoznik (Importer)

Kratak opis: Uvoznik dodaje proizvod iz ponude u postojeći kontejner.

Preduslovi:

- Korisnik je prijavljen sa IMPORTER ulogom
- Postoji kontejner sa statusom DRAFT
- Postoji bar jedna ponuda dostupna uvozniku

Glavni tok:

1. Uvoznik otvara stranicu sa ponudama. (APUSO)
2. Uvoznik klikne dugme **"Add to container"** pored željenog proizvoda. (APUSO)
3. Sistem otvara modal za dodavanje u kontejner. (IA)
4. Uvoznik bira kontejner i unosi količinu. (APUSO)
5. Uvoznik potvrđuje dodavanje klikom na **"Add"**. (APUSO)
6. Sistem šalje zahtev API-ju. (SO)
7. API proverava token, ulogu i status kontejnera. (SO)
8. API dodaje stavku u kontejner. (SO)
9. Sistem osvežava prikaz kontejnera. (IA)

Alternativni tokovi:

7.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)

7.2: Ukoliko korisnik nije IMPORTER — sistem vraća grešku: *"Forbidden."* (IA)

7.3: Ukoliko kontejner nije u statusu DRAFT — sistem vraća grešku: *"Container already finalized."* (IA)

7.4: Ukoliko kontejner ne postoji — sistem vraća grešku: *"Container not found."* (IA)

Postuslovi:

- Stavka je uspešno dodata u kontejner, ili je uvoznik obavešten o neuspehu.

SK12: Obrisi stavku iz kontejnera

Glavne uloge (akteri): Uvoznik (Importer)

Kratak opis: Uvoznik uklanja proizvod iz kontejnera koji je u statusu DRAFT.

Preduslovi:

- Korisnik je prijavljen sa IMPORTER ulogom
- Kontejner je u statusu DRAFT
- Kontejner sadrži bar jednu stavku

Glavni tok:

1. Uvoznik otvara detalje kontejnera. (APUSO)
2. Uvoznik klikne dugme **"Remove"** pored željene stavke. (APUSO)
3. Sistem prikazuje dijalog za potvrdu: *"Are you sure you want to remove this item from container?"* (IA)
4. Uvoznik potvrđuje uklanjanje. (APUSO)
5. Sistem šalje zahtev API-ju. (SO)
6. API proverava token, ulogu i status kontejnera. (SO)
7. API briše stavku iz kontejnera. (SO)
8. Sistem osvežava prikaz kontejnera. (IA)

Alternativni tokovi:

- 3.1: Ukoliko uvoznik odustane — sistem zatvara dijalog bez brisanja. (IA)
- 6.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)
- 6.2: Ukoliko korisnik nije IMPORTER — sistem vraća grešku: *"Forbidden."* (IA)
- 6.3: Ukoliko kontejner nije u statusu DRAFT — sistem vraća grešku: *"Cannot modify finalized container."* (IA)
- 6.4: Ukoliko kontejner ne postoji — sistem vraća grešku: *"Container not found."* (IA)
- 7.1: Ukoliko stavka ne postoji — sistem vraća grešku: *"Item not found."* (IA)

Postuslovi:

- Stavka je uspešno uklonjena iz kontejnera, ili je uvoznik obavešten o neuspehu.

SK13: Finalizuj kontejner

Glavne uloge (akteri): Uvoznik (Importer)

Kratak opis: Uvoznik finalizuje kontejner čime se onemogućava dalje dodavanje ili brisanje stavki.

Preduslovi:

- Korisnik je prijavljen sa IMPORTER ulogom
- Kontejner je u statusu DRAFT
- Kontejner je popunjen najmanje 50%

Glavni tok:

1. Uvoznik otvara detalje kontejnera. (APUSO)
2. Uvoznik klikne dugme **"Finalize container"**. (APUSO)
3. Sistem prikazuje dijalog za potvrdu: *"Are you sure you want to finalize this container? You will not be able to add more items."* (IA)
4. Uvoznik potvrđuje finalizaciju. (APUSO)
5. Sistem šalje zahtev API-ju. (SO)
6. API proverava token, ulogu i status kontejnera. (SO)
7. API proverava da li je kontejner popunjen najmanje 50%. (SO)
8. API menja status kontejnera u FINALIZED. (SO)
9. Sistem osvežava prikaz kontejnera. (IA)

Alternativni tokovi:

- 3.1: Ukoliko uvoznik odustane — sistem zatvara dijalog bez finalizacije. (IA)
- 6.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)
- 6.2: Ukoliko korisnik nije IMPORTER — sistem vraća grešku: *"Forbidden."* (IA)
- 6.3: Ukoliko kontejner nije u statusu DRAFT — sistem vraća grešku: *"Container already finalized."* (IA)
- 6.4: Ukoliko kontejner ne postoji — sistem vraća grešku: *"Container not found."* (IA)
- 7.1: Ukoliko kontejner nije popunjen najmanje 50% — sistem vraća grešku: *"Container must be at least 50% filled to finalize."* (IA)

Postuslovi: Kontejner je uspešno finalizovan i više se ne može menjati, ili je uvoznik obavешten o neuspehu.

SK14: Uporedi ponude proizvoda

Glavne uloge (akteri): Uvoznik (Importer)

Kratak opis: Uvoznik poredi između 2 i 4 proizvoda različitih dobavljača sa kojima sarađuje.

Preduslovi:

- Korisnik je prijavljen sa IMPORTER ulogom
- Uvoznik ima aktivne saradnje sa dobavljačima
- Postoje ponude dostupne uvozniku

Glavni tok:

1. Uvoznik otvara stranicu sa ponudama. (APUSO)
2. Uvoznik klikne dugme **"Compare"** pored željenih proizvoda. (APUSO)
3. Sistem preusmerava uvoznika na stranicu za poređenje. (IA)
4. Sistem učitava podatke o izabranim proizvodima. (SO)
5. Sistem prikazuje tabelu sa poređenjem — slika, šifra, kategorija, cena, širina, visina, dubina, opis i datum kreiranja. (IA)
6. Sistem zelenom bojom ističe najnižu vrednost za cenu i dimenzije. (IA)
7. Uvoznik opciono menja valutu iz padajućeg menija. (APUSO)
8. Sistem konvertuje cene u izabranu valutu u realnom vremenu. (SO)

Alternativni tokovi:

- 4.1: Ukoliko token ne postoji — sistem vraća grešku: *"Unauthorized."* (IA)
- 4.2: Ukoliko korisnik nije IMPORTER — sistem vraća grešku: *"Forbidden."* (IA)
- 4.3: Ukoliko nisu izabrani proizvodi — sistem prikazuje grešku: *"No products selected for comparison."* (IA)
- 4.4: Ukoliko je izabrano manje od 2 ili više od 4 proizvoda — sistem vraća grešku: *"You must compare between 2 and 4 products."* (IA)
- 4.5: Ukoliko proizvodi ne postoje ili uvoznik nema saradnju sa tim dobavljačem — sistem vraća grešku: *"No matching offers found."* (IA)
- 4.6: Ukoliko dođe do greške na serveru — sistem vraća grešku: *"Internal server error."* (IA)

7.1: Ukoliko kurs valute nije dostupan — sistem prikazuje cene u EUR. (IA)

Postuslovi:

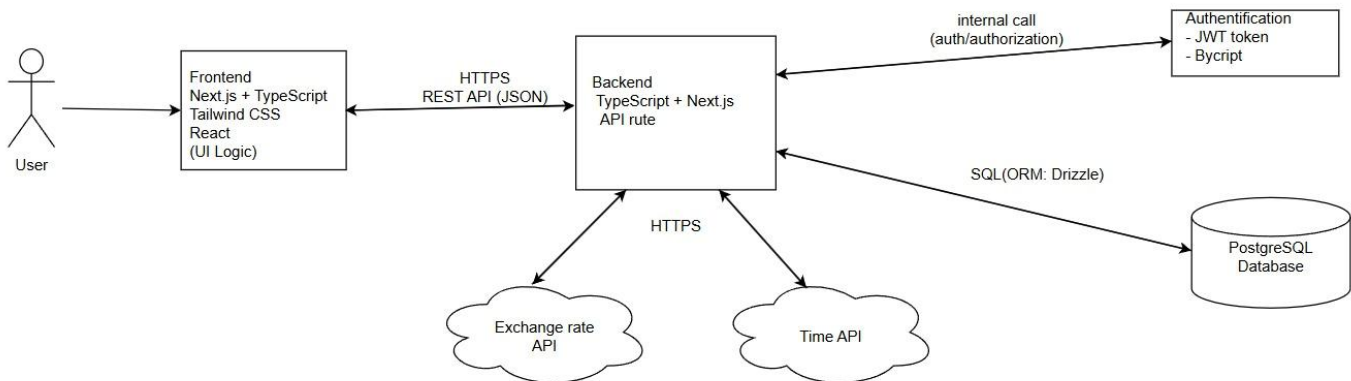
- Uvoznik uspešno poredi proizvode i dobija pregled sa istaknutim najboljim vrednostima, ili je obavešten o neuspehu.

2.2. Arhitektura aplikacije

Aplikacija je zasnovana na klijent–server arhitekturi. Korisnik, odnosno administrator, koristi frontend deo aplikacije koji je odgovoran za prikaz korisničkog interfejsa i unos podataka. Kada je potrebno dohvatiti ili izmeniti podatke, frontend upućuje zahteve backend REST API-ju koristeći HTTPS protokol.

Backend obrađuje pristigle zahteve, vrši validaciju ulaznih podataka i primenjuje definisana poslovna pravila. U slučajevima kada je neophodan rad sa podacima, backend komunicira sa PostgreSQL bazom podataka putem SQL upita, gde čita ili ažurira odgovarajuće zapise. Nakon obrade, backend generiše odgovor u JSON formatu i prosleđuje ga frontendu, koji zatim prikazuje rezultate korisniku.

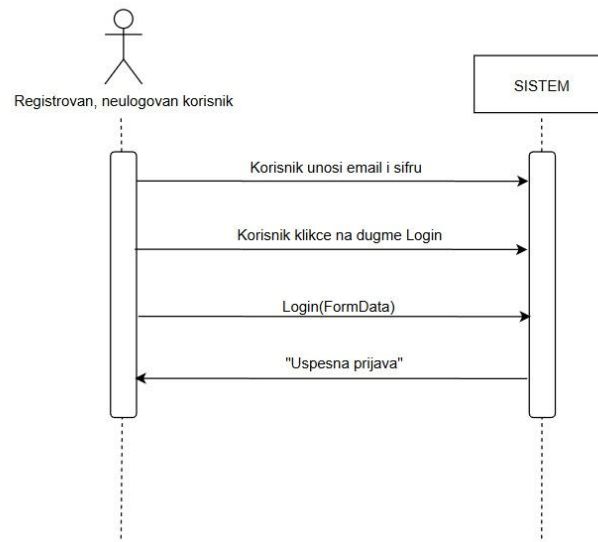
Za funkcionalnosti koje zahtevaju autentikaciju, frontend uz svaki zaštićeni zahtev šalje JWT token. Backend, kroz Auth modul, proverava validnost tokena, identifikuje korisnika i njegovu ulogu, a potom primenjuje RBAC mehanizam (kontrolu pristupa na osnovu uloga). Na ovaj način, određene funkcije dostupne su isključivo ovlašćenim korisnicima.



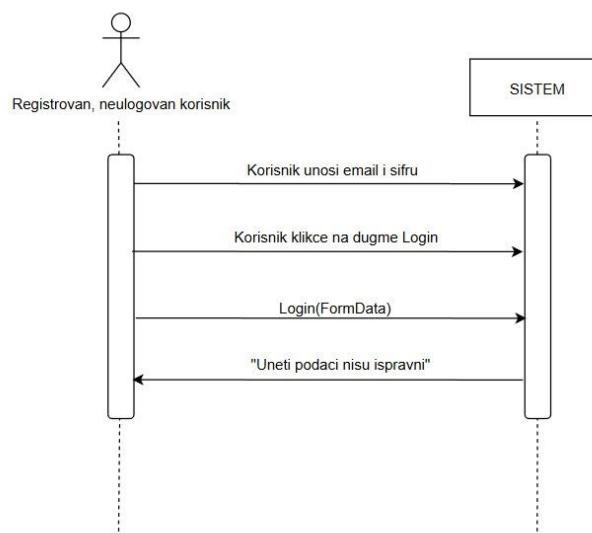
2.3. Opis procesa slučajeva korišćenja (Dijagrami sekvenci)

SK1 – Prijavi korisnika

Glavni scenario:

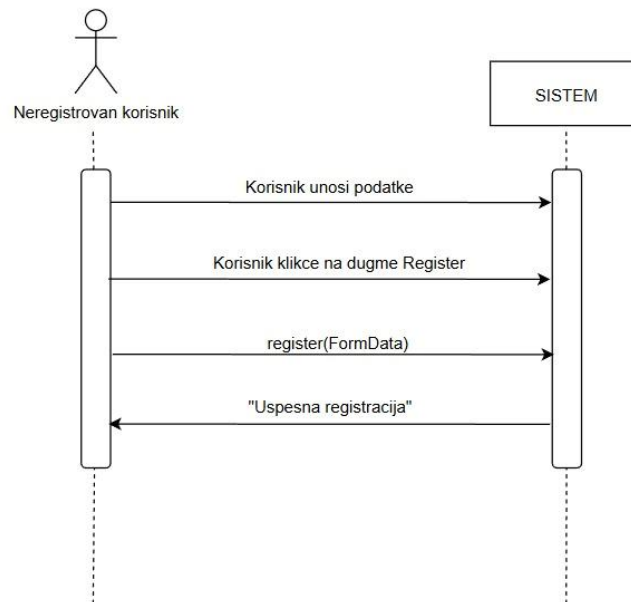


Alternativni scenario:

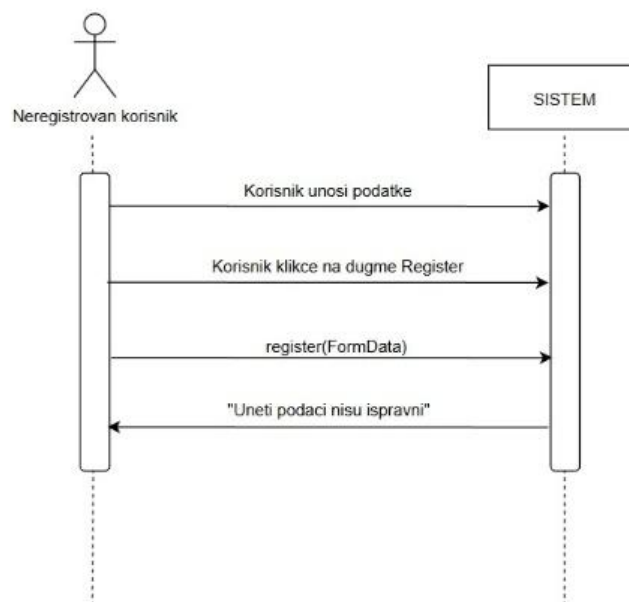


SK2 – Registruj korisnika

Glavni scenario:

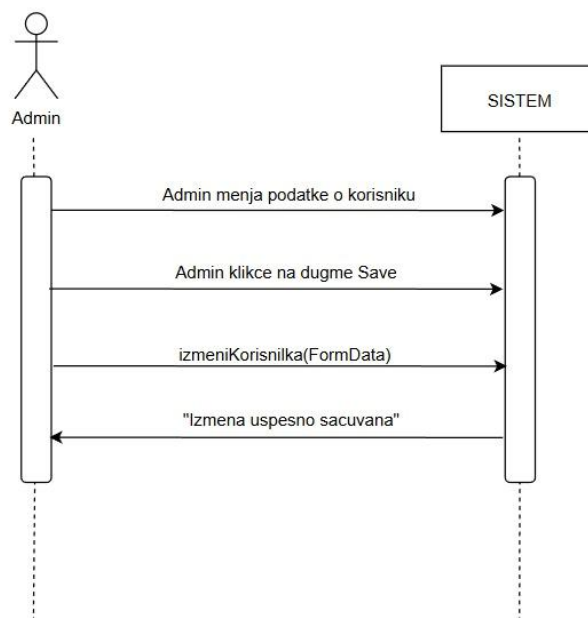


Alternativni scenario:

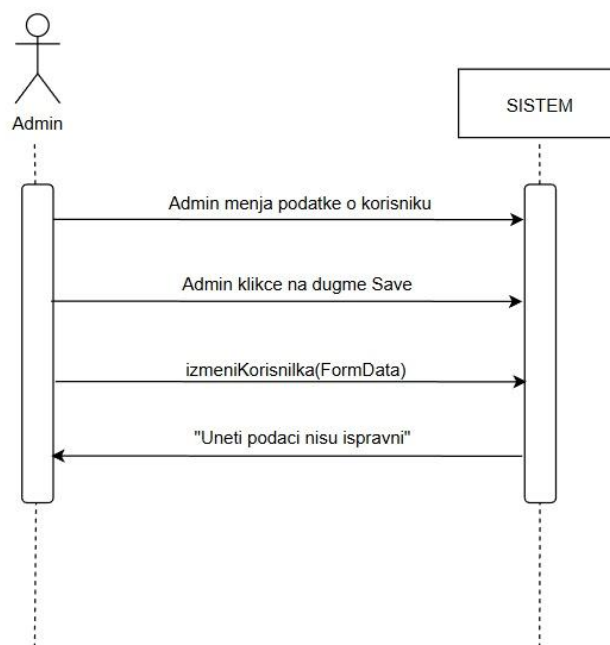


SK3 – Izmeni korisnika

Glavni scenario:

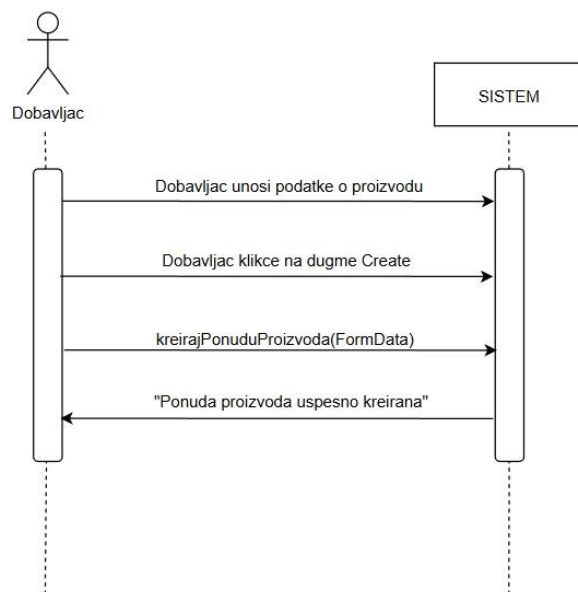


Alternativni scenario:

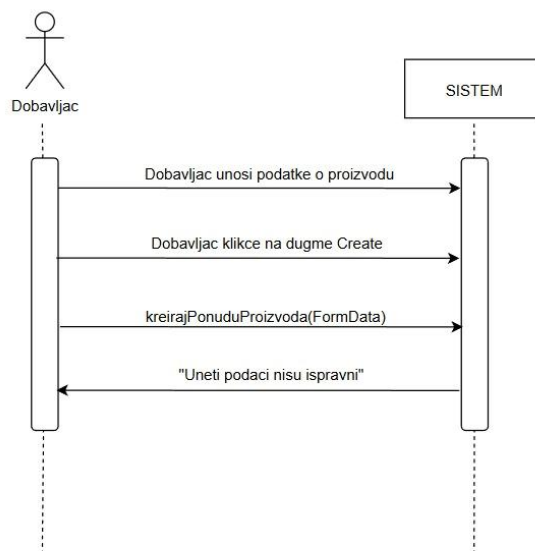


SK6 - Kreiraj ponudu proizvoda

Glavni scenario:

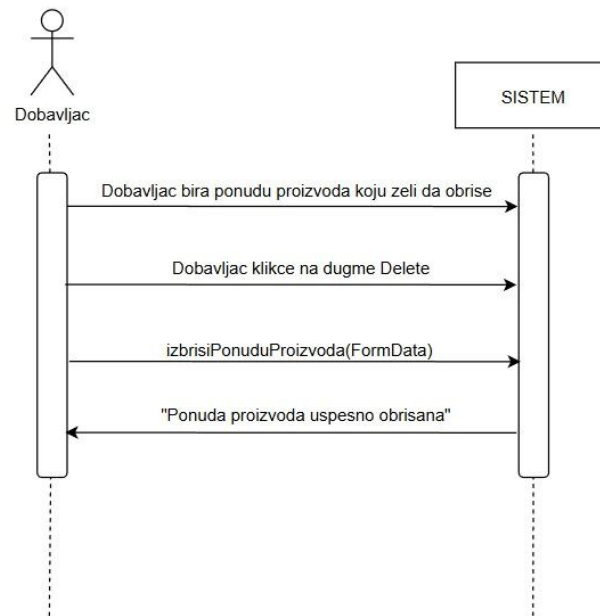


Alternativni scenario:



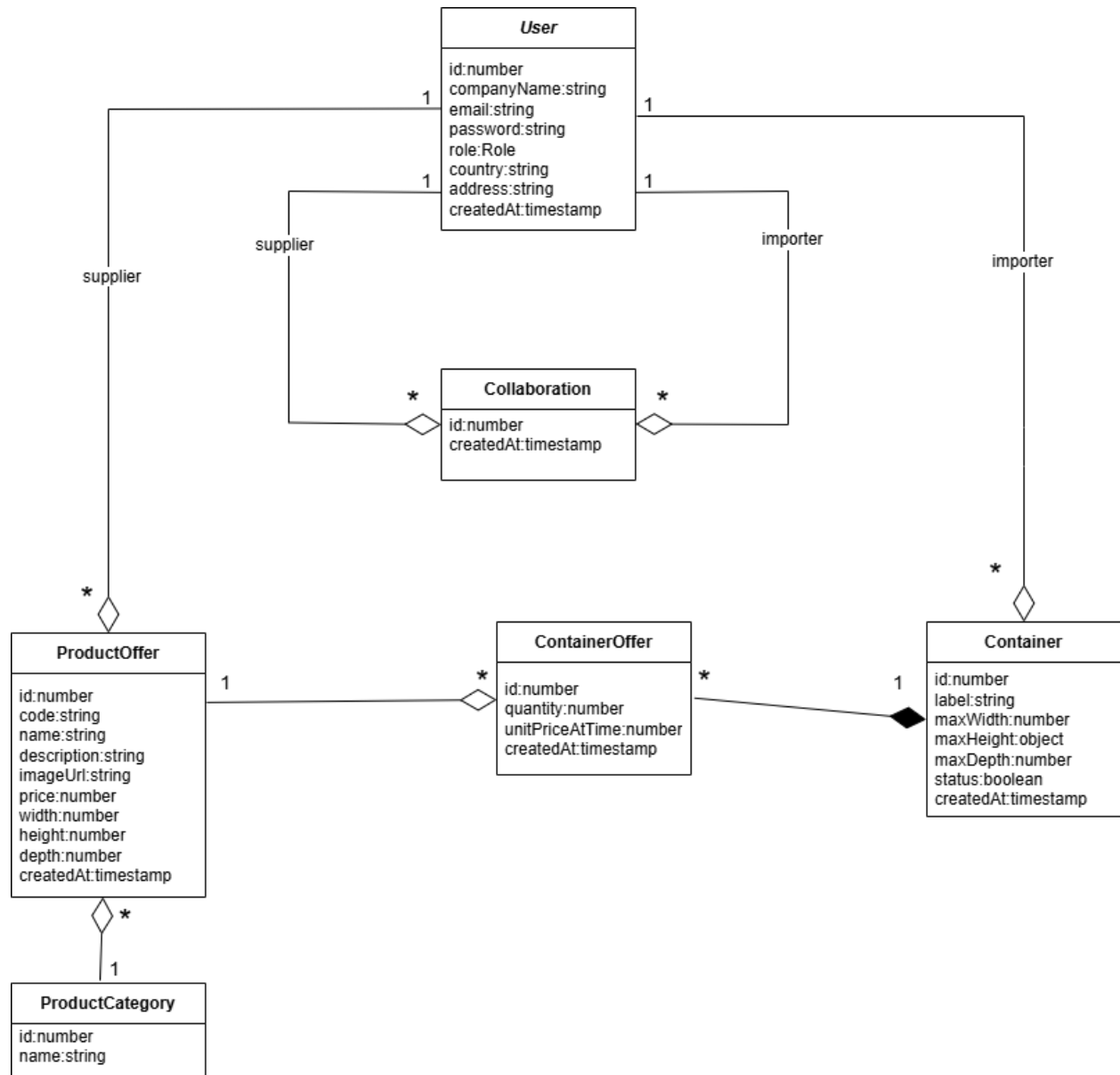
SK7 – Obrisí ponudu proizvoda

Glavni scenario:



2.4. Model podataka

Dijagram klasa



Relazioni model

USER(userId, companyName, email, password, role, country, address, createdAt)

PRODUCT_CATEGORY(categoryId, name)

PRODUCT_OFFER(offerId, *supplierId*, *categoryId*, code, name, description, imageUrl, price, width, height, depth, createdAt)

COLLABORATION(collaborationId, *supplierId*, *importerId*, createdAt)

CONTAINER(containerId, *importerId*, label, maxWidth, maxHeight, maxDepth, status, createdAt)

CONTAINER_OFFER(itemId, *containerId*, *productOfferId*, quantity, unitPriceAtTime, createdAt)

3. Predlog tehnologija koje će biti korišćene

3.1. Frontend

Za razvoj frontend dela aplikacije korišćen je **React** uz **TypeScript**. React je izabran zbog svoje komponentne arhitekture koja omogućava jednostavnu ponovnu upotrebu UI komponenti, lakše održavanje i bolju organizaciju koda. Ovakav pristup je posebno pogodan za aplikacije sa kompleksnim korisničkim interfejsom, kao što je sistem za pregled ponuda, upravljanje kontejnerima i rad sa velikom količinom podataka.

TypeScript je korišćen kako bi se povećala pouzdanost aplikacije kroz statičku proveru tipova, što smanjuje broj grešaka u radu sa podacima kao što su cene, dimenzije proizvoda i kapaciteti kontejnera. Time se poboljšava kvalitet koda i olakšava rad u timu.

Tailwind CSS omogućava brzo i efikasno stilizovanje korisničkog interfejsa pomoću unapred definisanih utility klasa, bez potrebe za pisanjem velike količine custom CSS-a. Ovakav pristup olakšava održavanje dizajna, obezbeđuje konzistentan izgled aplikacije i ubrzava razvoj responzivnog interfejsa.

3.2. Backend

Backend deo aplikacije razvijen je korišćenjem Next.js okvira, koji pored frontend funkcionalnosti omogućava i implementaciju serverske logike kroz API rute. Ovakav pristup omogućava jasno razdvajanje odgovornosti i efikasnu komunikaciju sa bazom podataka.

Backend je zadužen za:

- obradu poslovne logike aplikacije,
- upravljanje korisnicima i njihovim ulogama (uvoznik, dobavljač, administrator),
- autentifikaciju i autorizaciju korisnika,
- validaciju ulaznih podataka,
- upravljanje ponudama proizvoda,
- kreiranje i proveru kontejnera u skladu sa definisanim ograničenjima.

Serverska logika je realizovana kroz Next.js API rute, koje omogućavaju implementaciju REST-podobnih servisa bez potrebe za dodatnim backend framework-om kao što je Express. Ovakav način rada pojednostavljuje arhitekturu sistema i smanjuje složenost projekta.

Za rad sa bazom podataka korišćen je Drizzle ORM, koji obezbeđuje tipsku sigurnost, jasno definisane modele i jednostavno kreiranje upita. Drizzle omogućava lako mapiranje relacionog modela baze na objekte u aplikaciji, kao i sigurno izvršavanje CRUD operacija.

Asinhroni model rada serverske strane omogućava efikasno rukovanje većim brojem istovremenih zahteva, što je posebno važno u sistemu gde više uvoznika i dobavljača koristi aplikaciju u isto vreme.

3.3. Baza podataka

Za skladištenje podataka korišćena je **PostgreSQL** relacionalna baza podataka.

PostgreSQL je izabran zbog svoje stabilnosti, pouzdanosti i snažne podrške za rad sa kompleksnim relacionim strukturama podataka.

Relacione baze su posebno pogodne za ovaj projekat jer omogućavaju jasno modelovanje odnosa između:

- uvoznika,
- dobavljača,
- vrsta proizvoda,
- ponuda proizvoda,
- kontejnera i stavki kontejnera.

PostgreSQL obezbeđuje integritet podataka, podršku za transakcije i napredne upite, što je ključno za sprečavanje grešaka prilikom kreiranja kontejnera i obrade porudžbina.

3.4. Integracije

Aplikacija koristi **REST API komunikaciju** između frontend i backend dela sistema. Na ovaj način frontend aplikacija dobija podatke o ponudama, korisnicima i kontejnerima u realnom vremenu.

3.5. DevOps

Za potrebe razvoja i održavanja sistema predviđena je upotreba **Docker-a**, koji omogućava izolaciju okruženja i jednostavno pokretanje aplikacije na različitim sistemima bez dodatnih konfiguracija.

Za verzionisanje koda i automatizaciju procesa build-a i testiranja koristi se **GitHub Actions** što omogućava:

- automatsko testiranje koda,
- bržu isporuku novih verzija aplikacije.

4. Prikaz implementacije i korisničkog interfejsa

4.1. Prikaz delova koda sa objašnjenjima

1. Src-app-admin-collaborations

```
"use client";

import { useEffect, useState } from "react";

type CollaborationRow = {
  id: string;
  importerId: string;
  supplierId: string;
  createdAt: string | Date;

  importerEmail: string;
  importerCompanyName: string | null;

  supplierEmail: string;
  supplierCompanyName: string | null;
};
```

“use client” oznacava da je ovo Client Component u Next.js-u. Ovo znaci da se kod izvršava u browseru i može da koristi useEffect, useState, fetch iz klijenta i interaktivnost.

CollaborationRow predstavlja TypeScript tip koji opisuje saradnju. Svaka saradnja ima id, id uvoznika, id dobavljača, datum kreiranja, email i naziv kompanija uvoznika i dobavljača.

```
function label(email: string, companyName: string | null) {
  const name = companyName?.trim();
  return name ? `${name} (${email})` : email;
}
```

Ova funkcija formira prikaz imena u tabeli. Ako postoji naziv firme prikazuje se zajedno sa email-om, ako ne postoji naziv firme prikazuje se samo email.

```
export default function AdminCollaborationsPage() {
  const [rows, setRows] = useState<CollaborationRow[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
```

Ovo je stranica koja prikazuje sve saradnje. "Rows" - lista saradnji, "loading" - da li se podaci ucitavaju, "error" - eventualna greska.

```
useEffect(() => {
  let cancelled = false;

  (async () => {
    try {
      setLoading(true);
      setError(null);

      const res = await fetch("/api/admin/collaborations", {
        method: "GET",
        credentials: "include",
        cache: "no-store",
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(data?.error || `Neuspešno (HTTP ${res.status})`);
      }

      if (!cancelled) setRows(Array.isArray(data) ? data : []);
    } catch (e: any) {
      if (!cancelled) setError(e?.message ?? "Greška pri učitavanju.");
    } finally {
      if (!cancelled) setLoading(false);
    }
  })();

  return () => {
    cancelled = true;
  };
}, []);
```

Ovaj kod se izvršava jednom kada se komponenta učita. Poziva API, ako API vrati grešku baca error, ako je sve u redu postavlja promenljivu "rows", a ako se komponenta unmount-uje sprečava update state-a pomoću cancelled promenljive. Stranica ima 4 moguća stanja: loading, error, nema saradnji(prazna tabela) i postoje saradnje(prikazuju se).

2. Src-app-api-admin-collaborations

```
async function requireAdmin(): Promise<Claims | NextResponse> {
  const token = (await cookies()).get(AUTH_COOKIE)?.value;
  if (!token) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  try {
    const claims = verifyAuthToken(token);
    if (claims.role !== "ADMIN") {
      return NextResponse.json({ error: "Forbidden" }, { status: 403 });
    }
    return claims;
  } catch {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
}
```

Ova funkcija proverava da li je korisnik admin. Uzima auth cookie i ako nema tokena vraca "401 Unauthorized". Verifikuje JWT token i ako uloga nije "admin" vraca "403 Forbidden", a ako je sve u redu vraca "claims".

```
function getPgCode(e: any): string | undefined {
  return (
    e?.code ??
    e?.cause?.code ??
    e?.originalError?.code ??
    e?.cause?.originalError?.code
  );
}

function isDuplicateCollabError(e: any): boolean {
  const code = getPgCode(e);
  if (code === "23505") return true;

  const msg = String(
    e?.message ?? e?.cause?.message ?? e?.detail ?? e?.cause?.detail ?? ""
  ).toLowerCase();

  return msg.includes("duplicate key") || msg.includes("unique constraint");
}
```

Ovo su pomocne funkcije za PostgreSQL. Prva funkcija izvlaci Postgre error kod iz razlicitih mogucih mesta u error objektu. Druga funkcija proverava da li vec postoji saradnja izmedju uvoznika i dobavljacka.

```

export async function GET() {
  const admin = await requireAdmin();
  if (admin instanceof NextResponse) return admin;

  const importer = alias(users, "importer");
  const supplier = alias(users, "supplier");

  const data = await db
    .select({
      id: collaborations.id,
      importerId: collaborations.importerId,
      supplierId: collaborations.supplierId,
      createdAt: collaborations.createdAt,

      importerEmail: importer.email,
      importerCompanyName: importer.companyName,

      supplierEmail: supplier.email,
      supplierCompanyName: supplier.companyName,
    })
    .from(collaborations)
    .innerJoin(importer, eq(importer.id, collaborations.importerId))
    .innerJoin(supplier, eq(supplier.id, collaborations.supplierId))
    .orderBy(desc(collaborations.createdAt));

  return NextResponse.json(data);
}

```

GET metoda(vraca sve saradnje) prvo proverava da li je korisnik admin i zatim poziva SQL upit koji vraca id saradnje, id i email uvoznika i dobavljacka, kao i datum kreiranja saradnje. Na kraju salje dobijeni niz saradnji na frontend.

```

export async function POST(req: Request) {
  const admin = await requireAdmin();
  if (admin instanceof NextResponse) return admin;

  const body = (await req.json().catch(() => null)) as Body | null;
  const importerId = body?.importerId?.trim();
  const supplierId = body?.supplierId?.trim();

  if (!importerId || !supplierId) {
    return NextResponse.json(
      { error: "Importer i supplier su obavezni." },
      { status: 400 }
    );
  }

  if (importerId === supplierId) {
    return NextResponse.json(
      { error: "Importer i supplier ne mogu biti isti user." },
      { status: 400 }
    );
  }
}

```

POST metoda(kreira novu saradnju) isto prvo proverava da li je korisnik admin i zatim parsira body. Sledece proverava da li su uneti i uvoznik i dobavljac kao i da li su ista osoba. Ako nesto od ta dva ne valja vraca gresku 400.

```

const pair = await db
  .select({ id: users.id, role: users.role })
  .from(users)
  .where(inArray(users.id, [importerId, supplierId]));

const importer = pair.find((u) => u.id === importerId);
const supplier = pair.find((u) => u.id === supplierId);

if (!importer) {
  return NextResponse.json({ error: "Importer ne postoji." }, { status: 400 });
}
if (!supplier) {
  return NextResponse.json({ error: "Supplier ne postoji." }, { status: 400 });
}

if (importer.role !== "IMPORTER") {
  return NextResponse.json(
    { error: "Izabrani korisnik nije importer." },
    { status: 400 }
  );
}

if (supplier.role !== "SUPPLIER") {
  return NextResponse.json(
    { error: "Izabrani korisnik nije supplier." },
    { status: 400 }
  );
}

```

Proverava se da li oba korisnika postoje. Ako neki od ta dva ne postoji vraća gresku 400. Zatim se proverava da li su ta dva korisnika zapravo uvoznik i dobavljač, ako nisu vraća se ista greska.

```

try {
  await db.insert(collaborations).values({ importerId, supplierId });
  return NextResponse.json({ ok: true }, { status: 201 });
} catch (e: any) {
  if (isDuplicateCollabError(e)) {
    return NextResponse.json(
      { error: "Već postoji saradnja između izabranog importera i supplier-a." },
      { status: 409 }
    );
  }

  console.error("DB ERROR:", e);
  console.error("PG CODE:", getPgCode(e));

  return NextResponse.json(
    { error: "Došlo je do greške na serveru." },
    { status: 500 }
  );
}
}

```

Na kraju se vrši unos u bazu. Proverava se da li već postoji ista saradnja u bazi, ako postoji hvatamo tu gresku i ispisuje se poruka.

3. Src-app-admin-users

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { useRouter } from "next/navigation";
import UpdateUserModal, { UpdateUserPayload, AdminUser } from "@/components/UpdateUserModal";

type Filter = "ALL" | "IMPORTER" | "SUPPLIER";

export default function AdminUsersPage() {
  const router = useRouter();

  const [users, setUsers] = useState<AdminUser[]>([]);
  const [filter, setFilter] = useState<Filter>("ALL");
  const [loading, setLoading] = useState(true);
  const [editingUser, setEditingUser] = useState<AdminUser | null>(null);
```

Ovo je stranica koja omogućava adminu da ima pregled svih korisnika, filtrira ih po ulozi i azurira njihove podatke preko odgovarajuće komponente. "Users" - svi korisnici iz baze, "filter" - trenutni filter za pretraživanje, "loading" - da li se podaci učitavaju, "editingUser" - koji korisnik je trenutno otvoren u komponenti.


```

useEffect(() => {
  let cancelled = false;

  (async () => {
    try {

      const meRes = await fetch("/api/auth/me", {
        credentials: "include",
        cache: "no-store",
      });
      const meData = await meRes.json().catch(() => ({}));

      if (!meRes.ok || meData?.user?.role !== "ADMIN") {
        router.replace("/dashboard");
        return;
      }

      const res = await fetch("/api/admin/users", {
        credentials: "include",
        cache: "no-store",
      });

      const data = await res.json().catch(() => []);
      if (!cancelled) setUsers(Array.isArray(data) ? data : []);
    } catch {
      router.replace("/dashboard");
    } finally {
      if (!cancelled) setLoading(false);
    }
  })();

  return () => {
    cancelled = true;
  };
});

```

UseEffect prvo proverava da li je ulogovani korisnik admin. Ako korisnik nije ulogovan ili nije admin radi redirect na dashboard. Zatim se ucitavaju korisnici i postavlja se vrednost "users" pomocu setUsers(...). Ako dodje do greske opet se radi redirect.

```

const filteredUsers = useMemo(() => {
  return filter === "ALL" ? users : users.filter((u) => u.role === filter);
}, [users, filter]);

```

Filtriranje korisnika – ako je filter "ALL" prikazuju se svi korisnici, ako je "IMPORTER" prikazuju se uvoznici, a ako je "SUPPLIERS" prikazuju se dobavljači.

```

async function saveUser(userId: string, data: UpdateUserPayload) {
  const res = await fetch(`/api/admin/users/${userId}`, {
    method: "PUT",
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      email: data.email,
      companyName: data.companyName || null,
      country: data.country || null,
      address: data.address || null,
    }),
  });

  const payload = await res.json().catch(() => ({}));

  if (!res.ok) {
    throw new Error(payload?.error ?? "Update nije uspeo");
  }

  setUsers((prev) => prev.map((u) => (u.id === userId ? payload : u)));
  setEditingUser(null);
}

if (loading) {
  return (
    <main className="min-h-screen flex items-center justify-center bg-gray-100">
      <div className="text-gray-700">Loading users...</div>
    </main>
  );
}

const selectedUser = editingUser;

```

SetUser() je funkcija koja služi za izmenu podataka o korisniku i prosledjuje se odgovarajućoj komponenti. Salje podatke izabranog korisnika na backend, ako API ne vrati grešku, na kraju update-uje samo jednog korisnika u state-u i zatvara komponentu.

4. Src-app-api-admin-users

```
import { db } from "@db";
import { users } from "@db/schema/users";
import { inArray, asc } from "drizzle-orm";

export interface AdminUserDto {
  id: string;
  email: string;
  role: "ADMIN" | "IMPORTER" | "SUPPLIER";
  companyName: string | null;
  country: string | null;
  address: string | null;
  createdAt: Date;
}

export async function GET() {
  const data: AdminUserDto[] = await db
    .select({
      id: users.id,
      email: users.email,
      role: users.role,
      companyName: users.companyName,
      country: users.country,
      address: users.address,
      createdAt: users.createdAt,
    })
    .from(users)
    .where(inArray(users.role, ["IMPORTER", "SUPPLIER"]))
    .orderBy(asc(users.createdAt));
  return Response.json(data);
}
```

AdminUserDto je tip koji definiše strukturu podataka koje API vraća. GET metoda vraća niz svih korisnika bez admina.

5. Src-app-api-admin-users-[id]

```
import { db } from "@db";
import { users } from "@db/schema/users";
import { and, eq, inArray } from "drizzle-orm";

type UpdateUserBody = {
  email?: string;
  companyName?: string | null;
  country?: string | null;
  address?: string | null;

  role?: unknown;
  passHash?: unknown;
  id?: unknown;
};
```

Ovde definisemo sta frontend moze da salje. Role, passHash i id su unknown, cime se sprecava da neko menja njihove vrednosti.

```
function isUuid(v: string) {
  return /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i.test(
    v
  );
}
```

Ovom funkcijom proveravamo da li je uuid validan.

```

export async function PUT(
  req: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;

  if (!id || !isUuid(id)) {
    return Response.json({ error: "Neispravan id" }, { status: 400 });
  }

  let body: UpdateUserBody;
  try {
    body = (await req.json()) as UpdateUserBody;
  } catch {
    return Response.json({ error: "Neispravan JSON" }, { status: 400 });
  }
}

```

PUT metoda. Prvo se proverava da li je id validan, a zatim se parsira json body. Ukoliko id ili json nije validan vraća se greska.

```

const updateData: Partial<{
  email: string;
  companyName: string | null;
  country: string | null;
  address: string | null;
}> = {};

if (typeof body.email === "string") {
  const email = body.email.trim();
  if (!email) {
    return Response.json({ error: "Email je obavezan" }, { status: 400 });
  }
  updateData.email = email;
}

if (body.companyName === null || typeof body.companyName === "string") {
  const v = body.companyName;
  updateData.companyName = v === null ? null : v.trim() || null;
}

if (body.country === null || typeof body.country === "string") {
  const v = body.country;
  updateData.country = v === null ? null : v.trim() || null;
}

if (body.address === null || typeof body.address === "string") {
  const v = body.address;
  updateData.address = v === null ? null : v.trim() || null;
}

```

Dinamički se postavljaju samo validna polja i proverava se da li email, naziv kompanije, država i adresa nisu prazni stringovi. Ako jesu onda se postavlja null vrednost.

```

if (Object.keys(updateData).length === 0) {
  return Response.json(
    { error: "Nema validnih polja za update" },
    { status: 400 }
  );
}

try {
  const updated = await db
    .update(users)
    .set(updateData)
    .where(
      and(
        eq(users.id, id),
        inArray(users.role, ["IMPORTER", "SUPPLIER"])
      )
    )
    .returning({
      id: users.id,
      email: users.email,
      role: users.role,
      companyName: users.companyName,
      country: users.country,
      address: users.address,
      createdAt: users.createdAt,
    });
}

```

Proverava se da li ima validnih polja, i vrši se promena u bazi, pri čemu se ne dozvoljava promena podataka admin korisnika. Funkcija vraća update-ovanog korisnika.

```

    if (updated.length === 0) {
      return Response.json(
        { error: "Korisnik nije pronadjen (ili nije importer/supplier)" },
        { status: 404 }
      );
    }

    return Response.json(updated[0]);
  } catch (err: any) {
    const msg = typeof err?.message === "string" ? err.message : "Greska";

    if (msg.toLowerCase().includes("duplicate") || msg.toLowerCase().includes("unique")) {
      return Response.json({ error: "Email je vec zauzet" }, { status: 409 });
    }

    return Response.json({ error: "Neuspesan update" }, { status: 500 });
  }
}

```

Ukoliko nista nije update-ovano to znaci da taj korisnik ne postoji ili je admin i vraca se greska. Ako pokusamo da unesemo mejl koji vec u upotrebi vraca se greska.

6. Src-app-importer/offers

```
"use client";

import { useEffect, useMemo, useState } from "react";
import Sidebar from "@components/Sidebar";

type ImporterOfferDto = {
  id: string;

  categoryId: string;
  categoryName: string;

  code: string;
  name: string;
  description: string | null;
  imageUrl: string;
  price: string;
  width: number;
  height: number;
  depth: number;
  createdAt: string;

  supplierId: string;
  supplierEmail: string;
  supplierCompanyName: string | null;
};
```

ImporterOfferDto je tip podataka koji sadrži podatke o ponudi dobavljača, kategoriji proizvoda i dobavljaču.


```

export default function ImporterOffersPage() {
  const [offers, setOffers] = useState<ImporterOfferDto[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [query, setQuery] = useState("");

  async function load() {
    setLoading(true);
    setError(null);

    try {
      const res = await fetch("/api/importer/offers", {
        method: "GET",
        credentials: "include",
        cache: "no-store",
      });

      const data = await res.json().catch(() => null);

      if (!res.ok) {
        throw new Error(data?.error ?? `GET failed (${res.status})`);
      }

      setOffers((data ?? []) as ImporterOfferDto[]);
    } catch (e: any) {
      setError(e?.message ?? "Greška pri učitavanju ponuda");
    } finally {
      setLoading(false);
    }
  }
}

```

“Offers” - sve ponude iz baze, “loading” - da li se stranica učitava, “error” - potencijalna greska, “query” - tekst pretrage. Load funkcija učitava podatke o ponudama iz baze preko backenda. Ukoliko dodje do greske ona se i vraca.

```

useEffect(() => {
  load();
}, []);

// Search filter, pre grupisanja
const filtered = useMemo(() => {
  const q = query.trim().toLowerCase();
  if (!q) return offers;

  return offers.filter((o) => {
    return (
      o.code.toLowerCase().includes(q) ||
      o.name.toLowerCase().includes(q) ||
      (o.description ?? "").toLowerCase().includes(q) ||
      (o.categoryName ?? "").toLowerCase().includes(q) ||
      supplierLabel(o).toLowerCase().includes(q)
    );
  });
}, [offers, query]);

```

UseMemo() radi filtriranje pre grupisanja. Moze se pretrazivati po kodu, nazivu, opisu, kategoriji i dobavljacu. Pretraga je case insensitive i trim-ovana. Ako nema pretrage vraca sve ponude.

```

// Grupisanje po supplier-u
const grouped = useMemo(() => {
  const map = new Map<
    string,
    { supplierId: string; supplierName: string; items: ImporterOfferDto[] }
  >();

  for (const o of filtered) {
    const key = o.supplierId;
    if (!map.has(key)) {
      map.set(key, {
        supplierId: o.supplierId,
        supplierName: supplierLabel(o),
        items: [],
      });
    }
    map.get(key)!.items.push(o);
  }

  // sort grupe po imenu supplier-a
  const groups = Array.from(map.values()).sort((a, b) =>
    a.supplierName.localeCompare(b.supplierName)
  );

  // sort ponude unutar grupe po datumu (najnovije prvo)
  for (const g of groups) {
    g.items.sort(
      (a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
    );
  }

  return groups;
}, [filtered]);

```

Ovde useMemo() radi grupisanje po dobavljaču. Za svaku ponudu prvo proverava da li grupa postoji, ako ne postoji kreira je i na kraju ponudu ubacuje u odgovarajuću grupu. Zatim se vrši sortiranje grupa po imenu dobavljača, a onda se i unutar grupe ponude sortiraju po datumu kreiranja (najnovije prvo).

7. Src-app-api-importer/offers

```
import { db } from "@db";
import { productOffers } from "@db/schema/productOffers";
import { productCategories } from "@db/schema/productCategories";
import { collaborations } from "@db/schema/collaborations";
import { users } from "@db/schema/users";

import { desc, eq, and } from "drizzle-orm";
import { alias } from "drizzle-orm/pg-core";

import { cookies } from "next/headers";
import { AUTH_COOKIE, verifyAuthToken } from "@lib/auth";

export async function GET() {
  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;
  if (!token) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const { sub: userId, role } = verifyAuthToken(token);
  if (role !== "IMPORTER") {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  const supplierUser = alias(users, "supplier_user");
```

Vrsi se autentifikacija tako sto se uzima JWT token iz cookie-a i verifikuje se da li je korisnik uvoznik. Ako token ne postoji ili korisnik nije uvoznik, vraca se greska. Na kraju se kreira alias za tabelu users.

```

const data = await db
  .select({
    id: productOffers.id,

    categoryId: productOffers.categoryId,
    categoryName: productCategories.name,

    code: productOffers.code,
    name: productOffers.name,
    description: productOffers.description,
    imageUrl: productOffers.imageUrl,
    price: productOffers.price,
    width: productOffers.width,
    height: productOffers.height,
    depth: productOffers.depth,
    createdAt: productOffers.createdAt,

    supplierId: productOffers.supplierId,
    supplierEmail: supplierUser.email,
    supplierCompanyName: supplierUser.companyName,
  })
  .from(productOffers)
  .innerJoin(
    collaborations,
    and(
      eq(collaborations.importerId, userId),
      eq(collaborations.supplierId, productOffers.supplierId)
    )
  )
  .innerJoin(productCategories, eq(productOffers.categoryId, productCategories.id))
  .innerJoin(supplierUser, eq(supplierUser.id, productOffers.supplierId))
  .orderBy(desc(productOffers.createdAt));

return Response.json(data);
}

```

Ovo je glavni upit koji vraća sve ponude dobavljača sa kojima trenutno ulogovani uvoznik ima saradnje. Pored ponude upit vraća i kategoriju proizvoda, kao i uvoznika čija je ponuda. Na kraju se ponude sortiraju po datumu i šalju se na frontend.

8. Src-app-supplier

```
"use client";

import { useEffect, useMemo, useState } from "react";
import OfferCard, { SupplierOfferDto } from "@components/OfferCard";

type Role = "ADMIN" | "IMPORTER" | "SUPPLIER";

export default function SupplierOffersPage() {
  const [offers, setOffers] = useState<SupplierOfferDto[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [deletingId, setDeletingId] = useState<string | null>(null);
  const [query, setQuery] = useState("");

  const filtered = useMemo(() => {
    const q = query.trim().toLowerCase();
    if (!q) return offers;

    return offers.filter((o) => {
      return (
        o.code.toLowerCase().includes(q) ||
        o.name.toLowerCase().includes(q) ||
        (o.description ?? "").toLowerCase().includes(q) ||
        (o.categoryName ?? "").toLowerCase().includes(q)
      );
    });
  }, [offers, query]);
```

“offers” - lista ponuda dobavljača, “loading” - da li se stranica učitava, “error” - greska, “deletingID” - čuva id ponude koja se trenutno brise, “query” - tekst pretrage. UseMemo() filtrira ponude kada se promeni offers ili query. Filtriranje se može raditi po sifri, nazivu, opisu ili kategoriji. Ako je query prazan, vraćaju se sve ponude.

```

async function load() {
  setLoading(true);
  setError(null);

  try {
    const res = await fetch("/api/supplier/offers", {
      method: "GET",
      credentials: "include",
      cache: "no-store",
    });

    const data = await res.json().catch(() => null);

    if (!res.ok) {
      throw new Error(data?.error ?? `GET failed (${res.status})`);
    }

    setOffers((data ?? []) as SupplierOfferDto[]);
  } catch (e: any) {
    setError(e?.message ?? "Greška pri učitavanju ponuda");
  } finally {
    setLoading(false);
  }
}

```

Funkcija load() radi učitavanje ponuda preko backenda. Ako sve prodje kako treba postavlja vrednost offers na dobijen odgovor, u suprotnom se vraća greska.

```

async function handleDelete(id: string) {
  const ok = confirm("Da li sigurno želiš da obrišeš ovu ponudu?");
  if (!ok) return;

  const prev = offers;
  setDeletingId(id);
  setOffers((cur) => cur.filter((o) => o.id !== id));

  try {
    const res = await fetch(`/api/supplier/offers/${id}`, {
      method: "DELETE",
      credentials: "include",
    });

    const data = await res.json().catch(() => null);

    if (!res.ok) {
      throw new Error(data?.error ?? `DELETE failed (${res.status})`);
    }
  } catch (e: any) {
    setOffers(prev);
    alert(e?.message ?? "Greška pri brisanju");
  } finally {
    setDeletingId(null);
  }
}

```

Funkcija `handleDelete()` vrši brisanje ponude. Prvo traži potvrdu za brisanje od korisnika. Izabrana ponuda se prvo briše iz UI-a, pa se tek onda šalje zahtev na backend kako bi aplikacija bila brza. Ako prilikom brisanja backend vrati gresku radi se rollback.

9. Src-app-api-supplier

```
import { productOffers } from "@db/schema/productOffers";
import { desc, eq } from "drizzle-orm";
import { cookies } from "next/headers";
import { AUTH_COOKIE, verifyAuthToken } from "@lib/auth";
import { productCategories } from "@db/schema/productCategories";

export async function GET() {
  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;
  if (!token) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const { sub: userId, role } = verifyAuthToken(token);
  if (role !== "SUPPLIER") return Response.json({ error: "Forbidden" }, { status: 403 });

  const data = await db
    .select({
      id: productOffers.id,
      categoryId: productOffers.categoryId,
      categoryName: productCategories.name,
      code: productOffers.code,
      name: productOffers.name,
      description: productOffers.description,
      imageUrl: productOffers.imageUrl,
      price: productOffers.price,
      width: productOffers.width,
      height: productOffers.height,
      depth: productOffers.depth,
      createdAt: productOffers.createdAt,
    })
    .from(productOffers)
    .innerJoin(productCategories, eq(productOffers.categoryId, productCategories.id))
    .where(eq(productOffers.supplierId, userId))
    .orderBy(desc(productOffers.createdAt));

  return Response.json(data);
}
```

GET metoda se koristi za pristup ponudama dobavljača iz baze. Prvo se radi autentifikacija teko što se verifikuje JWT token. Ukoliko token ne postoji ili ulogovani korisnik nije dobavljač, vraća se greska. Zatim se vrši SQL upit nad bazom koji vraća podatke o ponudama koji se filtriraju tako da se dobavljaču prikazu samo njegove ponude. Ponude se i sortiraju po datumu(prvo najnovije).

```

export async function POST(req: Request) {
  try {

    const cookieStore = await cookies();
    const token = cookieStore.get(AUTH_COOKIE)?.value;

    if (!token) {
      return Response.json({ error: "Unauthorized" }, { status: 401 });
    }

    const { sub: userId, role } = verifyAuthToken(token);

    if (role !== "SUPPLIER") {
      return Response.json({ error: "Forbidden" }, { status: 403 });
    }

    const body = await req.json();

    const {
      categoryId,
      code,
      name,
      description,
      imageUrl,
      price,
      width,
      height,
      depth,
    } = body ?? {};
  }
}

```

POST metoda takodje prvo radi autentifikaciju kao sto je i u GET metodi. Koristi se za kreiranje nove ponude. Body zahteva se parsira u prikazane promenljive.

```

// Minimalna validacija - backend
if (
  !categoryId ||
  !code ||
  !name ||
  !imageUrl ||
  price === undefined ||
  width === undefined ||
  height === undefined ||
  depth === undefined
) {
  return Response.json(
    { error: "Missing required fields" },
    { status: 400 }
  );
}

// Insert u bazu
const inserted = await db
  .insert(productOffers)
  .values({
    supplierId: userId,
    categoryId,
    code,
    name,
    description: description ?? null,
    imageUrl,
    price,
    width,
    height,
    depth,
  })
  .returning({
    id: productOffers.id,
  });

```

Zatim se proverava da li su uneti svi potrebni podaci. Ukoliko nisu vraća se greska. Sledeći korak je ubacivanje nove ponude u bazu i vraća se id te ponude.

```
// Response
return Response.json(
  {
    id: inserted[0].id,
    message: "Offer created",
  },
  { status: 201 }
);
} catch (err) {
  console.error("POST /api/supplier/offers error:", err);
  return Response.json(
    { error: "Internal server error" },
    { status: 500 }
  );
}
}
```

Na kraju se salje odgovor na frontend.

10. Src-app-api-supplier-[id]

```
import { db } from "@db";
import { productOffers } from "@db/schema/productOffers";
import { and, eq } from "drizzle-orm";
import { cookies } from "next/headers";
import { AUTH_COOKIE, verifyAuthToken } from "@lib/auth";

type Role = "ADMIN" | "IMPORTER" | "SUPPLIER";

export async function DELETE(
  _req: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id: offerId } = await params;

  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;

  if (!token) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { sub: userId, role } = verifyAuthToken(token);

  if (role !== "SUPPLIER") {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }
}
```

DELETE metoda se koristi za brisanje ponude dobavljača. Prvo se, kao i ranije, vrši autentifikacija korisnika.

```
const deleted = await db
  .delete(productOffers)
  .where(and(eq(productOffers.id, offerId), eq(productOffers.supplierId, userId)))
  .returning({ id: productOffers.id });

if (deleted.length === 0) {
  return Response.json({ error: "Not found" }, { status: 404 });
}

return Response.json({ ok: true });
}
```

Zatim se vrši brisanje ponude iz baze. Ukoliko ponuda ne postoji ili ne pripada odgovarajućem dobavljaču, vraća se greška.

11. Middleware.ts

```
src > TS middleware.ts > ...
1  import { NextRequest, NextResponse } from "next/server";
2  import { jwtVerify } from "jose";
3
4  const AUTH_COOKIE = "auth";
5  type Role = "ADMIN" | "IMPORTER" | "SUPPLIER";
6
7  async function verifyInMiddleware(token: string) {
8    const secret = new TextEncoder().encode(process.env.JWT_SECRET!);
9    const { payload } = await jwtVerify(token, secret);
10
11    const sub = payload.sub;
12    const role = payload.role as Role | undefined;
13
14    if (!sub || !role) throw new Error("Invalid token");
15    return { sub, role };
16  }
17
18  export async function middleware(req: NextRequest) {
19    const { pathname } = req.nextUrl;
20
21    const token = req.cookies.get(AUTH_COOKIE)?.value;
22
23    // nije ulogovan
24    if (!token) {
25      return NextResponse.redirect(new URL("/login", req.url));
26    }
27
28    try {
29      const user = await verifyInMiddleware(token);
30
31      // ADMIN rute
32      if (pathname.startsWith("/admin") && user.role !== "ADMIN") {
33        return NextResponse.redirect(new URL("/dashboard", req.url));
34      }
35
36      // IMPORTER rute
37      if (
38        pathname.startsWith("/importer") &&
39        !["ADMIN", "IMPORTER"].includes(user.role)
40      ) {
41        return NextResponse.redirect(new URL("/dashboard", req.url));
42      }
43
44      // SUPPLIER rute
45      if (
46        pathname.startsWith("/supplier") &&
47        !["ADMIN", "SUPPLIER"].includes(user.role)
48      ) {
49        return NextResponse.redirect(new URL("/dashboard", req.url));
50      }
51
52      // sve dozvoljeno
53      return NextResponse.next();
54    } catch {
55      // token ne važi
56      return NextResponse.redirect(new URL("/login", req.url));
57    }
58  }
59
60  export const config = {
61    matcher: [
62      "/admin/:path*",
63      "/importer/:path*",
64      "/supplier/:path*",
65    ],
66  };
67
```

Middleware predstavlja centralni mehanizam za proveru da li je korisnik ulogovan i da li ima odgovarajuću ulogu za pristup određenim delovima sistema. Izvršava se pre učitavanja stranica koje pripadaju administrativnim ili korisničkim zonama aplikacije. Implementiran je u skladu sa Next.js mehanizmom i koristi JWT token.

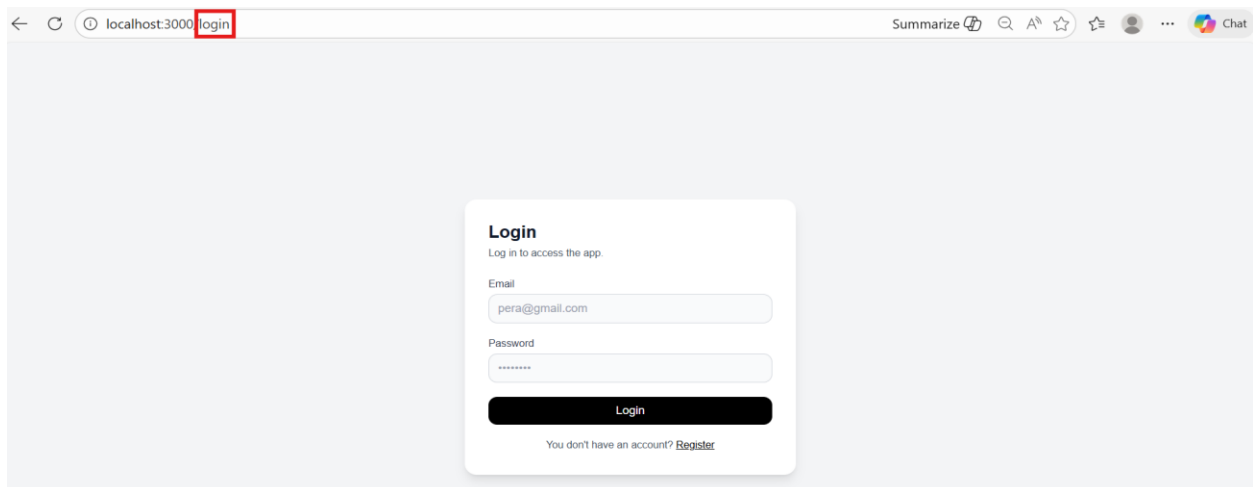
Middleware obavlja sledeće provere:

1. Da li korisnik poseduje autentifikacioni token.
2. Da li je token validan i pravilno potpisan.
3. Koja je uloga korisnika.
4. Da li korisnik sa tom ulogom ima pravo pristupa traženoj ruti.

Primer zaštite ruta (autorizacija putem middleware-a)

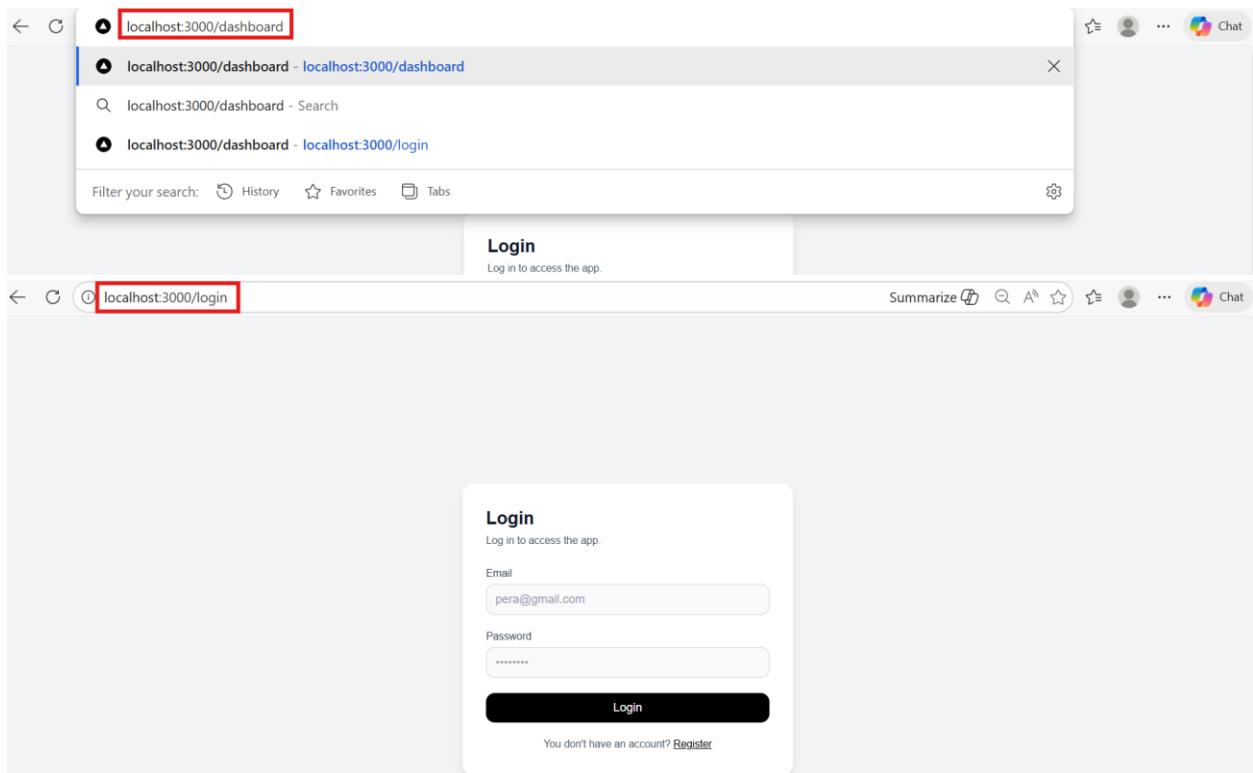
Sistem koristi middleware mehanizam kako bi sprečio neautorizovan pristup zaštićenim delovima aplikacije.

Ukoliko korisnik pokuša da ručno unese URL, na primer `/dashboard`, `/admin`, `/importer` ili `/supplier`, aplikacija prvo proverava da li postoji validan autentifikacioni token u kolačićima.



Ukoliko pokušamo da promenimo adresu iz **localhost:3000/login** u **localhost:3000/dashboard**, middleware neće dozvoliti pristup toj stranici jer korisnik nije autentifikovan, odnosno nema validan token.

Pošto token nije pronađen u kolačićima, korisnik se automatski preusmerava nazad na login stranicu. Na taj način sistem sprečava neovlašćen pristup zaštićenim delovima aplikacije.



12. Src-db-schema-collaborations

```
src > db > schema > TS collaborations.ts > collaborations > supplierId
1  import {
2    pgTable,
3    uuid,
4    timestamp,
5    unique,
6  } from "drizzle-orm/pg-core";
7  import { users } from "../users";
8
9  export const collaborations = pgTable(
10   "collaborations",
11   {
12     id: uuid("id").defaultRandom().primaryKey(),
13
14     importerId: uuid("importer_id")
15       .notNull()
16       .references(() => users.id, { onDelete: "restrict" }),
17
18     supplierId: uuid("supplier_id")
19       .notNull()
20       .references(() => users.id, { onDelete: "restrict" }),
21
22     createdAt: timestamp("created_at", { withTimezone: true })
23       .defaultNow()
24       .notNull(),
25   },
26   (t) => ({
27     importerSupplierUnique: unique(
28       "collaborations_importer_supplier_unique"
29     ).on(t.importerId, t.supplierId),
30   })
31 );
32
33
```

Kod definiše collaborations tabelu koja povezuje importere i dobavljače (many-to-many veza između korisnika). Tabela ima unique constraint koji sprečava duplo povezivanje istog para importer-supplier, a onDelete: restrict sprečava brisanje korisnika ako ima aktivne saradnje.

13. Src-db-schema-containerOffer

```
src > db > schema > TS containerOffer.ts > containerOffers
1  import {
2    pgTable,
3    uuid,
4    timestamp,
5    integer,
6    numeric,
7    uniqueIndex,
8  } from "drizzle-orm/pg-core";
9
10 import { containers } from "../containers";
11 import { productOffers } from "../productOffers";
12
13 export const containerOffers = pgTable(
14   "container_offer",
15   {
16     id: uuid("id").defaultRandom().primaryKey(),
17
18     containerId: uuid("container_id")
19       .notNull()
20       .references(() => containers.id, { onDelete: "cascade" }),
21
22     offerId: uuid("offer_id")
23       .notNull()
24       .references(() => productOffers.id, { onDelete: "restrict" }),
25
26     quantity: integer("quantity").notNull(),
27
28     unitPriceAtTime: numeric("unit_price_at_time", {
29       precision: 12,
30       scale: 2,
31     }).notNull(),
32
33     createdAt: timestamp("created_at", { withTimezone: true })
34       .defaultNow()
35       .notNull(),
36   },
37   (t) => ({
38     uniqContainerOffer: uniqueIndex("uq_container_offer").on(
39       t.containerId,
40       t.offerId
41     ),
42   })
43 );
```

Kod definiše containerOffer tabelu koja predstavlja stavke narudžbine - povezuje kontejnere sa proizvodima iz ponuda, čuvajući količinu i cenu u trenutku naručivanja. onDelete: cascade znači da ako se obriše kontejner, brišu se i njegove stavke, dok restrict sprečava brisanje ponude ako je korišćena u kontejneru; uniqueIndex sprečava dodavanje istog proizvoda dva puta u isti kontejner.

14. Src-db-schema-containers

```
src > db > schema > TS containers.ts > containers
1  import { pgTable, uuid, varchar, timestamp, real } from "drizzle-orm/pg-core";
2  import { users } from "../users";
3
4  export const containers = pgTable("container", {
5    id: uuid("id").defaultRandom().primaryKey(),
6
7    importerId: uuid("importer_id")
8      .notNull()
9      .references(() => users.id, { onDelete: "cascade" }),
10
11    label: varchar("label", { length: 128 }),
12
13    maxWidth: real("max_width").notNull(),
14    maxHeight: real("max_height").notNull(),
15    maxDepth: real("max_depth").notNull(),
16
17    status: varchar("status", { length: 32 })
18      .notNull()
19      .default("DRAFT"),
20
21    createdAt: timestamp("created_at", { withTimezone: true })
22      .defaultNow()
23      .notNull(),
24
25  });
26
27
```

Kod definiše containers tabelu koja čuva informacije o kontejnerima koje kreira importer - svaki kontejner ima labelu, maksimalne dimenzije (širina, visina, dubina) i status (default "DRAFT"). Kada se obriše importer (onDelete: cascade), automatski se brišu i svi njegovi kontejneri.

15. Src-db-schema-productCategories

```
src > db > schema > TS productCategories.ts > ...
1  import { pgTable, uuid, varchar } from "drizzle-orm/pg-core";
2
3  export const productCategories = pgTable("product_category", {
4    id: uuid("id").defaultRandom().primaryKey(),
5
6    name: varchar("name", { length: 180 }).notNull().unique(),
7  });
8
9
```

Kod definiše productCategories tabelu koja čuva nazive kategorija proizvoda (npr. "Elektronika", "Odeća"). Svaka kategorija ima jedinstveno ime (unique) što sprečava kreiranje duplikata.

16. Src-db-schema-users

```
src > db > schema > TS users.ts > ...
1  import { pgTable, uuid, varchar, text, timestamp, pgEnum } from "drizzle-orm/pg-core";
2
3  export const userRoleEnum = pgEnum("user_role", [
4    "ADMIN",
5    "IMPORTER",
6    "SUPPLIER",
7  ]);
8
9  export const users = pgTable("users", {
10    id: uuid("id").defaultRandom().primaryKey(),
11
12    email: varchar("email", { length: 255 }).notNull().unique(),
13    passHash: text("pass_hash").notNull(),
14
15    companyName: varchar("company_name", { length: 255 }),
16    country: varchar("country", { length: 100 }),
17    address: text("address"),
18
19    role: userRoleEnum("role").notNull(),
20
21    createdAt: timestamp("created_at", { withTimezone: true })
22      .defaultNow()
23      .notNull(),
24  });
25
```

Kod definiše users tabelu koja čuva korisnike sistema sa njihovim podacima (email, lozinka, ime kompanije, država, adresa) i ulogom (ADMIN, IMPORTER, SUPPLIER). Email mora biti jedinstven (unique), a uloga se bira iz predefinisano enum-a; createdAt automatski beleži vreme registracije korisnika.

17. Src-db-schema- productOffer

```
src > db > schema > TS productOffers.ts > [E] productOffers > supplierId
1  import {
2    pgTable,
3    uuid,
4    varchar,
5    text,
6    timestamp,
7    numeric,
8    real,
9  } from "drizzle-orm/pg-core";
10 import { users } from "../users";
11 import { productCategories } from "../productCategories";
12
13 export const productOffers = pgTable("product_offer", {
14   id: uuid("id").defaultRandom().primaryKey(),
15   //
16   supplierId: uuid("supplier_id")
17     .notNull()
18     .references(() => users.id, { onDelete: "cascade" }),
19   categoryId: uuid("category_id")
20     .notNull()
21     .references(() => productCategories.id, { onDelete: "restrict" }),
22   code: varchar("code", { length: 64 }).notNull(),
23   name: varchar("name", { length: 255 }).notNull(),
24   description: text("description"),
25   imageUrl: text("image_url").notNull(),
26   price: numeric("price", { precision: 12, scale: 2 }).notNull(),
27   width: real("width").notNull(),
28   height: real("height").notNull(),
29   depth: real("depth").notNull(),
30   createdAt: timestamp("created_at", { withTimezone: true })
31     .defaultNow()
32     .notNull(),
33 });
41
```

Kod definiše productOffer tabelu koja čuva ponude proizvoda koje kreiraju dobavljači - svaki proizvod ima šifru, naziv, opis, sliku, cenu i dimenzije (širina, visina, dubina). Kada se obriše dobavljač (onDelete: cascade), brišu se i njegovi proizvodi, dok se kategorija ne može obrisati (restrict) ako postoje proizvodi u njoj.

Compare funkcionalnost

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { useRouter, useSearchParams } from "next/navigation";

type ImporterOfferDto = {
  id: string;
  categoryId: string;
  categoryName: string;
  code: string;
  name: string;
  description: string | null;
  imageUrl: string;
  price: string;
  width: number;
  height: number;
  depth: number;
  createdAt: string;
  supplierId: string;
  supplierEmail: string;
  supplierCompanyName: string | null;
};
```

Ovde se vrsi definicija tipa objekta "offer" koji sadrzi podatke o kategoriji proizvoda, proizvodu, kao i dobavljkacu koji nudi taj proizvod.

```

const SUPPORTED_CURRENCIES = ["EUR", "RSD", "BAM", "MKD", "HUF", "BGN", "RON"];

export default function ComparePage() {
  const router = useRouter();
  const searchParams = useSearchParams();

  const [offers, setOffers] = useState<ImporterOfferDto[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const [rates, setRates] = useState<Record<string, number>>({});
  const [selectedCurrency, setSelectedCurrency] = useState("EUR");

  const idsParam = searchParams.get("ids");

  useEffect(() => {
    async function load() {
      if (!idsParam) {
        setError("No products selected for comparison.");
        setLoading(false);
        return;
      }
    }
  });
}

```

Prvo se definišu podrzane valute, a zatim i state promenljive kao sto su lista proizvoda koji se porede, učitavanje, greska, kursna lista i izabrana valuta. UseEffect učitava proizvode i ako nema izabranih vraca gresku.


```

    try {
      const res = await fetch(
        `/api/importer/offers/compare?ids=${idsParam}`,
        { credentials: "include", cache: "no-store" }
      );

      const data = await res.json().catch(() => null);

      if (!res.ok) {
        throw new Error(data?.error ?? "Failed to load comparison.");
      }

      setOffers(data ?? []);
    } catch (e: any) {
      setError(e?.message ?? "Error loading comparison.");
    } finally {
      setLoading(false);
    }
  }
}

```

Zatim se vrši pozivanje api-ja i od backend-a se dobija lista proizvoda za poredjenje. Ukoliko podaci nisu dobri vraća se greška.

```

useEffect(() => {
  async function loadRates() {
    try {
      const res = await fetch("/api/exchange", {
        credentials: "include",
        cache: "no-store",
      });

      if (!res.ok) return;

      const data = await res.json();
      setRates(data.rates || {});
    } catch {
      console.error("Exchange rate fetch failed");
    }
  }

  loadRates();
}, []);

function convert(price: number) {
  if (selectedCurrency === "EUR") return price;
  const rate = rates[selectedCurrency];
  if (!rate) return price;
  return price * rate;
}

```

UseEffect vraća kurseve valuta preko api-ja i smesta podatke u "rates". Zatim imamo funkciju za konverziju valute tako što pretvara cenu iz EUR u izabranu valutu.

```

const minValues = useMemo(() => {
  if (offers.length === 0) return null;

  return {
    price: Math.min(...offers.map(o => convert(Number(o.price)))),
    width: Math.min(...offers.map(o => o.width)),
    height: Math.min(...offers.map(o => o.height)),
    depth: Math.min(...offers.map(o => o.depth)),
  };
}, [offers, selectedCurrency, rates]);

function highlight(value: number, min?: number) {
  if (min === undefined) return "";
  return value === min ? "text-green-600 font-bold" : "";
}

function formatCurrency(value: number) {
  return new Intl.NumberFormat(undefined, {
    style: "currency",
    currency: selectedCurrency,
    maximumFractionDigits: 2,
  }).format(value);
}

```

UseMemo racuna najmanje vrednosti za poređenje, a funkcija "highlight" ce te vrednosti podebljati i obojiti u zeleno. Na kraju imamo funkciju za formatiranje valute.

Api-compare ruta

```
export async function GET(req: Request) {
  try {
    const cookieStore = await cookies();
    const token = cookieStore.get(AUTH_COOKIE)?.value;

    if (!token) {
      return Response.json({ error: "Unauthorized" }, { status: 401 });
    }

    const { sub: userId, role } = verifyAuthToken(token);

    if (role !== "IMPORTER") {
      return Response.json({ error: "Forbidden" }, { status: 403 });
    }

    const { searchParams } = new URL(req.url);
    const idsParam = searchParams.get("ids");

    if (!idsParam) {
      return Response.json(
        { error: "No product IDs provided." },
        { status: 400 }
      );
    }

    const ids = idsParam.split(",").map((id) => id.trim()).filter(Boolean);
```

Get funkcija prvo vrsi autentifikaciju tako sto verifikuje JWT token iz cookie-ja i proverava se da li je korisnik uvoznik. Zatim se uzimaju parametri iz url-a i ako nema id-jeva proizvoda vraca se greska.

```
if (ids.length < 2 || ids.length > 4) {
  return Response.json(
    { error: "You must compare between 2 and 4 products." },
    { status: 400 }
  );
}

const supplierUser = alias(users, "supplier_user");
```

Ovde se definisu ogranicenja broja proizvoda koji se mogu porediti i pravi se alias za tabelu users.

```

const data = await db
  .select({
    id: productOffers.id,

    categoryId: productOffers.categoryId,
    categoryName: productCategories.name,

    code: productOffers.code,
    name: productOffers.name,
    description: productOffers.description,
    imageUrl: productOffers.imageUrl,
    price: productOffers.price,
    width: productOffers.width,
    height: productOffers.height,
    depth: productOffers.depth,
    createdAt: productOffers.createdAt,

    supplierId: productOffers.supplierId,
    supplierEmail: supplierUser.email,
    supplierCompanyName: supplierUser.companyName,
  })
  .from(productOffers)
  .innerJoin(
    collaborations,
    and(
      eq(collaborations.importerId, userId),
      eq(collaborations.supplierId, productOffers.supplierId)
    )
  )
)

```

Zatim sledi sql upit kojim se dobijaju potrebni podaci o proizvodu, vrsti proizvoda kao i o dobavljaču koji nudi proizvod.

```

        .innerJoin(
            productCategories,
            eq(productOffers.categoryId, productCategories.id)
        )
        .innerJoin(
            supplierUser,
            eq(supplierUser.id, productOffers.supplierId)
        )
        .where(inArray(productOffers.id, ids))
        .orderBy(desc(productOffers.createdAt));

    if (data.length < 1) {
        return Response.json(
            { error: "No matching offers found." },
            { status: 404 }
        );
    }

    return Response.json(data);
} catch (err) {
    console.error(err);
    return Response.json(
        { error: "Internal server error." },
        { status: 500 }
    );
}
}

```

Importer može videti ponude samo od dobavljača sa kojima ima saradnju. Proizvodi se sortiraju po vremenu kreiranja opadajuće. Na kraju se podaci vraćaju frontendu.

Api-exchange ruta

```
export async function GET() {
  try {
    const cookieStore = await cookies();
    const token = cookieStore.get(AUTH_COOKIE)?.value;

    if (!token) {
      return Response.json({ error: "Unauthorized" }, { status: 401 });
    }

    verifyAuthToken(token);

    const response = await fetch(
      "https://api.exchangerate-api.com/v4/latest/EUR",
      { cache: "no-store" }
    );

    if (!response.ok) {
      return Response.json(
        { error: "Failed to fetch exchange rates" },
        { status: 500 }
      );
    }

    const data = await response.json();
```

Ovo je API endpoint koji vraća kursne liste valuta. GET funkcija prvo verifikuje token, a zatim se poziva eksterni API koji vraća kurs valuta u odnosu na eur.

```
    return Response.json({
      base: data.base,
      rates: data.rates,
      date: data.date,
    });
  } catch (err) {
    console.error("Exchange rates error:", err);

    return Response.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}
```

Formatira se odgovor i šalje se frontendu.

API-time ruta

```
export async function GET() {
  try {
    const cookieStore = await cookies();
    const token = cookieStore.get(AUTH_COOKIE)?.value;

    if (!token) {
      return Response.json({ error: "Unauthorized" }, { status: 401 });
    }

    const { role } = verifyAuthToken(token);

    if (!role) {
      return Response.json({ error: "Forbidden" }, { status: 403 });
    }

    const response = await fetch(
      "https://timeapi.io/api/Time/current/zone?timeZone=Europe/Belgrade",
      { cache: "no-store" }
    );

    if (!response.ok) {
      return Response.json(
        { error: "Failed to fetch external time API" },
        { status: 500 }
      );
    }

    const data = await response.json();

    const externalDate = new Date(data.dateTime);
```

GET funkcija prvo vrši autentifikaciju, a onda poziva eksterni api koji vraća trenutno vreme u Beogradu.

```
    return Response.json({
      externalTime: externalDate.getTime(),
      serverNow: Date.now(),
    });
  } catch (err) {
    console.error("GET /api/time error:", err);

    return Response.json(
      { error: "Internal server error" },
      { status: 500 }
    );
  }
}
```

Zatim se formatira odgovor koji sadrži vreme sa api-ja i vreme servera, i šalje se na frontend.

Dashboard (prikaz datuma i vremena preko Time api)

```
useEffect(() => {
  async function initTime() {
    try {
      const res = await fetch("/api/time", {
        credentials: "include",
        cache: "no-store",
      });

      if (!res.ok) return;

      const data = await res.json();

      const externalTime = data.externalTime;
      const serverNow = data.serverNow;

      offsetRef.current = externalTime - serverNow;

      function updateTime() {
        const now = new Date(Date.now() + offsetRef.current);
        setCurrentDateTime(now);
      }

      updateTime();

      const now = new Date(Date.now() + offsetRef.current);
      const seconds = now.getSeconds();
      const delay = (60 - seconds) * 1000;

      timeoutRef.current = setTimeout(() => {
        updateTime();
        intervalRef.current = setInterval(updateTime, 60000);
      }, delay);
    } catch (err) {
      console.error("Time init failed");
    }
  }
}, [offsetRef, intervalRef, timeoutRef]);
```

Na frontendu useEffect prvo proverava da li su dobri podaci, a onda funkcija "updateTime" racuna ofset, tj. razliku izmedju vremena sa api-ja i vremena servera. Zatim se taj ofset dodaje na trenutno vreme kako bi se dobilo sinhronizovano vreme. Na svakih minut se vrsi azuriranje sata kako bi se promenilo vreme na korisnickom interfejsu.

Containers stranica

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import NewContainerModal from "@/components/NewContainerModal";

type ContainerDto = {
  id: string;
  label: string | null;
  maxWidth: number;
  maxHeight: number;
  maxDepth: number;
  status: string;
  createdAt: string;
};

export default function ImporterContainersPage() {
  const router = useRouter();

  const [statusFilter, setStatusFilter] =
    useState<"ALL" | "DRAFT" | "FINALIZED">("ALL");

  const [containers, setContainers] = useState<ContainerDto[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const [showNewModal, setShowNewModal] = useState(false);

  async function load() {
    setLoading(true);
    setError(null);
  }
}
```

Ovo je stranica za prikaz svih kontejnera. Prvo se vrši definisanje tipa podataka "ContainerDto" koji ima attribute id, naziv, maksimalna visina, sirina i dubina, status i vreme kreiranja. Zatim se definise state koji kontrolise koji kontejneri se prikazuju, kao i state-ovi koji cuvaju listu kontejnera, ucitavanje i gresku.

```

try {
  const res = await fetch("/api/importer/containers", {
    credentials: "include",
    cache: "no-store",
  });

  const data = await res.json().catch(() => null);

  if (!res.ok) {
    throw new Error(data?.error ?? "Failed to load containers");
  }

  setContainers(Array.isArray(data) ? data : []);
} catch (e: any) {
  setError(e?.message ?? "Greška");
} finally {
  setLoading(false);
}
}

useEffect(() => {
  load();
}, []);

const filteredContainers =
  statusFilter === "ALL"
    ? containers
    : containers.filter((c) => c.status === statusFilter);

```

Vrsi se funkcija za učitavanje kontejnera sa bekenda. Ako dobijeni podaci nisu dobri vraca se greska. Pre svakog poziva se vrsi reset stanja – postavlja se loading i brise se stara greska. Na kraju postoji opcija za filtriranje kontejnera u odnosu na stanje u kom se nalaze – all, draft, finalized.

API-Containers ruta

```
export async function GET(req: Request) {
  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;

  if (!token) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { sub: userId, role } = verifyAuthToken(token);

  if (role !== "IMPORTER") {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  const url = new URL(req.url);
  const statusFilter = url.searchParams.get("status");

  const whereCondition =
    statusFilter && (statusFilter === "DRAFT" || statusFilter === "FINALIZED")
      ? and(
          eq(containers.importerId, userId),
          eq(containers.status, statusFilter)
        )
      : eq(containers.importerId, userId);
```

GET funkcija prvo vrši autentifikaciju tako što verifikuje JWT token i proverava da li je korisnik uvoznik. Iz Url-a se uzima filter i proverava se da li je validan.

```
const data = await db
  .select({
    id: containers.id,
    label: containers.label,
    maxWidth: containers.maxWidth,
    maxHeight: containers.maxHeight,
    maxDepth: containers.maxDepth,
    status: containers.status,
    createdAt: containers.createdAt,
  })
  .from(containers)
  .where(whereCondition)
  .orderBy(desc(containers.createdAt));

return Response.json(data);
```

Zatim se vrši Sql upit i dobijeni kontejneri se sortiraju po vremenu kreiranja opadajuće.

```

export async function POST(req: Request) {
  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;

  if (!token) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { sub: userId, role } = verifyAuthToken(token);

  if (role !== "IMPORTER") {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  const body = await req.json();

  const width = Number(body.maxWidth);
  const height = Number(body.maxHeight);
  const depth = Number(body.maxDepth);

  if (!width || !height || !depth) {
    return Response.json(
      { error: "Invalid dimensions" },
      { status: 400 }
    );
  }
}

```

POST funkcija služi za kreiranje novog kontejnera. Prvo se vrši autentifikacija kao i kod get funkcije. Zatim se čita i parsira body zahteva. Nakon toga se proverava da li dimenzije postoje kao i da li su dimenzije unutar dozvoljenih vrednosti. Ako nešto od toga nije ispunjeno vraća se greška.

```

if (width < 2 || width > 3) {
  return Response.json(
    { error: "Width must be between 2.00 and 3.00 meters." },
    { status: 400 }
  );
}

if (height < 2 || height > 3) {
  return Response.json(
    { error: "Height must be between 2.00 and 3.00 meters." },
    { status: 400 }
  );
}

if (depth < 5 || depth > 13) {
  return Response.json(
    { error: "Depth must be between 5.00 and 13.00 meters." },
    { status: 400 }
  );
}

const [container] = await db
  .insert(containers)
  .values({
    importerId: userId,
    label: body.label || null,
    maxWidth: width,
    maxHeight: height,
    maxDepth: depth,
    status: "DRAFT",
  })
  .returning();

return Response.json(container);

```

Na kraju se vrši Sql upit za kreiranje kontejnera i rezultat se vraća na frontend.

Stranica za prikaz kontejnera

```
type ContainerData = {
  container: {
    id: string;
    label: string | null;
    status: string;
  };
  items: {
    id: string;
    name: string;
    quantity: number;
    price: string;
  }[];
  percentage: number;
  usedVolume: number;
  maxVolume: number;
};

export default function ContainerDetailsPage() {
  const params = useParams();
  const router = useRouter();

  const rawId = params?.id;
  const id = Array.isArray(rawId) ? rawId[0] : rawId;

  const [data, setData] = useState<ContainerData | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
```

Ovo je client stranica koja prikazuje detalje jednog kontejnera. Prvo se definise tip podataka za kontejner koji sadrzi podatke o kontejneru, listu proizvoda koji se u njemu nalaze, procenat popunjenosti kao i maksimalan kapacitet kontejnera.

```

async function load() {
  if (!id) return;

  try {
    setLoading(true);
    setError(null);

    const res = await fetch(`/api/importer/containers/${id}`, {
      credentials: "include",
      cache: "no-store",
    });

    const result = await res.json();

    if (!res.ok) {
      throw new Error(result?.error || "Failed to load container");
    }

    setData(result);
  } catch (err: any) {
    setError(err?.message || "Error loading container");
  } finally {
    setLoading(false);
  }
}

```

Funkcija "load" učitava podatke o kontejneru iz api-ja. Ako podaci nisu dobri vraća se greška.


```

async function finalize() {
  if (!id) return;

  const confirmed = window.confirm(
    "Are you sure you want to finalize this container?\n\nYou will not be able to add more items."
  );

  if (!confirmed) return;

  try {
    const res = await fetch(
      `/api/importer/containers/${id}/finalize`,
      {
        method: "POST",
        credentials: "include",
      }
    );

    const result = await res.json();

    if (!res.ok) {
      alert(result?.error || "Failed to finalize container.");
      return;
    }

    alert("Container successfully finalized.");
    load();
  } catch {
    alert("Something went wrong.");
  }
}

```

Funkcija “finalize” zaključuje kontejner kada korisnik završi sa unosom proizvoda. Prvo se traži potvrda korisnika da je završio sa unosom, a zatim se poziva api za finalizovanje kontejnera.

```

async function removeItem(itemId: string) {
  if (!id) return;

  const confirmed = window.confirm(
    "Are you sure you want to remove this item from container?"
  );

  if (!confirmed) return;

  try {
    const res = await fetch(
      `/api/importer/containers/${id}/items/${itemId}`,
      {
        method: "DELETE",
        credentials: "include",
      }
    );

    const result = await res.json();

    if (!res.ok) {
      alert(result?.error || "Failed to remove item.");
      return;
    }

    load();
  } catch {
    alert("Something went wrong.");
  }
}

```

Funkcija "removeItem" vrši brisanje proizvoda iz kontejnera. Prvo se traži potvrda korisnika da želi da obrise proizvod, a zatim se poziva api zadužen za brisanje.

```

useEffect(() => {
  if (id) load();
}, [id]);

const totalContainerValue = useMemo(() => {
  if (!data) return 0;

  return data.items.reduce((sum, item) => {
    return sum + Number(item.price) * item.quantity;
  }, 0);
}, [data]);

const totalUnits = useMemo(() => {
  if (!data) return 0;

  return data.items.reduce((sum, item) => {
    return sum + item.quantity;
  }, 0);
}, [data]);

const distinctProducts = useMemo(() => {
  if (!data) return 0;
  return data.items.length;
}, [data]);

```

UseEffect ucitava kontejner kada se promeni id. Prvi useMemo racuna ukupnu vrednost proizvoda u kontejneru. Drugi usememo racuna broj jedinica svih proizvoda u kontejneru. Treci useMemo racuna koliko razlicitih proizvoda postoji.

Api/importer/containers/id

```
export async function GET(req: Request) {  
  const cookieStore = await cookies();  
  const token = cookieStore.get(AUTH_COOKIE)?.value;  
  
  if (!token) {  
    return Response.json({ error: "Unauthorized" }, { status: 401 });  
  }  
  
  const { sub: userId, role } = verifyAuthToken(token);  
  
  if (role !== "IMPORTER") {  
    return Response.json({ error: "Forbidden" }, { status: 403 });  
  }  
  
  const url = new URL(req.url);  
  const containerId = url.pathname.split("/").pop();  
  
  if (!containerId) {  
    return Response.json({ error: "Invalid id" }, { status: 400 });  
  }  
  
  const [container] = await db  
    .select()  
    .from(containers)  
    .where(  
      and(  
        eq(containers.id, containerId),  
        eq(containers.importerId, userId)  
      )  
    );  
}
```

GET funkcija služi za dobijanje podataka o nekom kontejneru. Prvo vrši autentifikaciju kao i do sad, a zatim iz url-a uzima id kontejner čije podatke treba prikazati. Vrši se upit koji vraća odgovarajući kontejner.

```

if (!container) {
  return Response.json({ error: "Not found" }, { status: 404 });
}

const items = await db
  .select({
    id: containerOffers.id,
    quantity: containerOffers.quantity,
    name: productOffers.name,
    price: containerOffers.unitPriceAtTime,
    width: productOffers.width,
    height: productOffers.height,
    depth: productOffers.depth,
  })
  .from(containerOffers)
  .innerJoin(
    productOffers,
    eq(productOffers.id, containerOffers.offerId)
  )
  .where(eq(containerOffers.containerId, containerId));

const maxVolume =
  container.maxWidth *
  container.maxHeight *
  container.maxDepth;

let usedVolume = 0;

for (const item of items) {
  usedVolume +=
    (item.width / 100) *
    (item.height / 100) *
    (item.depth / 100) *
    item.quantity;
}

```

Kada je nadjen kontejner, vrsi se novi sql upit koji ucitava sve proizvode koji se nalaze u tom kontejneru. Zatim se racuna kapacitet kontejnera, kao i zapremina koju zauzimaju svi proizvodi u njemu.

```

const percentage =
  maxVolume === 0 ? 0 : (usedVolume / maxVolume) * 100;

return Response.json({
  container,
  items,
  usedVolume,
  maxVolume,
  percentage,
});

```

Na kraju se racuna procenat popunjenosti kontejnera i odgovor se salje na frontend.

Api/importer/containers/id/finalize

```
export async function POST(req: Request) {
  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;

  if (!token) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { sub: userId, role } = verifyAuthToken(token);

  if (role !== "IMPORTER") {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  const url = new URL(req.url);
  const containerId = url.pathname.split("/").slice(-2)[0];

  if (!containerId) {
    return Response.json({ error: "Invalid id" }, { status: 400 });
  }

  const [container] = await db
    .select()
    .from(containers)
    .where(
      and(
        eq(containers.id, containerId),
        eq(containers.importerId, userId)
      )
    );

  if (!container) {
    return Response.json({ error: "Container not found" }, { status: 404 });
  }
}
```

POST funkcija služi za finalizovanje kontejnera. Prvo se vrši autentifikacija kao i do sad, a zatim se uzima id kontejnera iz url-a. Vrši se sql upit koji vraća odgovarajući kontejner. Ako je loš id ili kontejner ne postoji vraća se greška.

```

if (container.status !== "DRAFT") {
  return Response.json(
    { error: "Container already finalized" },
    { status: 400 }
  );
}

const items = await db
  .select({
    quantity: containerOffers.quantity,
    width: productOffers.width,
    height: productOffers.height,
    depth: productOffers.depth,
  })
  .from(containerOffers)
  .innerJoin(
    productOffers,
    eq(productOffers.id, containerOffers.offerId)
  )
  .where(eq(containerOffers.containerId, containerId));

```

Kontejner se ne može finalizovati ako njegov status nije "DRAFT". Poziva se novi sql upit koji vraća sve proizvode koji se nalaze u kontejneru.

```

const maxVolume =
  container.maxWidth *
  container.maxHeight *
  container.maxDepth;

let usedVolume = 0;

for (const item of items) {
  usedVolume +=
    (item.width / 100) *
    (item.height / 100) *
    (item.depth / 100) *
    item.quantity;
}

const percentage =
  maxVolume === 0 ? 0 : (usedVolume / maxVolume) * 100;

if (percentage < 50) {
  return Response.json(
    { error: "Container must be at least 50% filled to finalize." },
    { status: 400 }
  );
}

await db
  .update(containers)
  .set({ status: "FINALIZED" })
  .where(eq(containers.id, containerId));

return Response.json({ success: true });

```

Racuna se kapacitet kontejnera, kao i zapremina koju zauzimaju svi proizvodi u njemu. Zatim se racuna procenat popunjenosti kontejnera i proverava se da li je on veci od 50. Na kraju se poziva sql upit koji azurira vrednost kontejnera i menja njegov status na "FINALIZED". Podatak o uspesnosti se vraca na frontend.

Api/importer/containers/id/items/itemId

```
export async function DELETE(req: Request) {
  const cookieStore = await cookies();
  const token = cookieStore.get(AUTH_COOKIE)?.value;

  if (!token) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { sub: userId, role } = verifyAuthToken(token);

  if (role !== "IMPORTER") {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  const url = new URL(req.url);
  const parts = url.pathname.split("/");

  const containerId = parts[parts.length - 3];
  const itemId = parts[parts.length - 1];

  if (!containerId || !itemId) {
    return Response.json({ error: "Invalid request" }, { status: 400 });
  }

  const [container] = await db
    .select()
    .from(containers)
    .where(
      and(
        eq(containers.id, containerId),
        eq(containers.importerId, userId)
      )
    );
};
```

DELETE funkcija služi za brisanje proizvoda iz kontejnera. Prvo se vrši autentifikacija i proveru uloge. Zatim se uzima id kontejnera i proizvoda za brisanje iz url-a. Ako neki id ne postoji vraća se greska. Poziva se sql upit koji vraća odgovarajući kontejner.

```

if (!container) {
  return Response.json({ error: "Container not found" }, { status: 404 });
}

if (container.status !== "DRAFT") {
  return Response.json(
    { error: "Cannot modify finalized container" },
    { status: 400 }
  );
}

const result = await db
  .delete(containerOffers)
  .where(
    and(
      eq(containerOffers.id, itemId),
      eq(containerOffers.containerId, containerId)
    )
  );

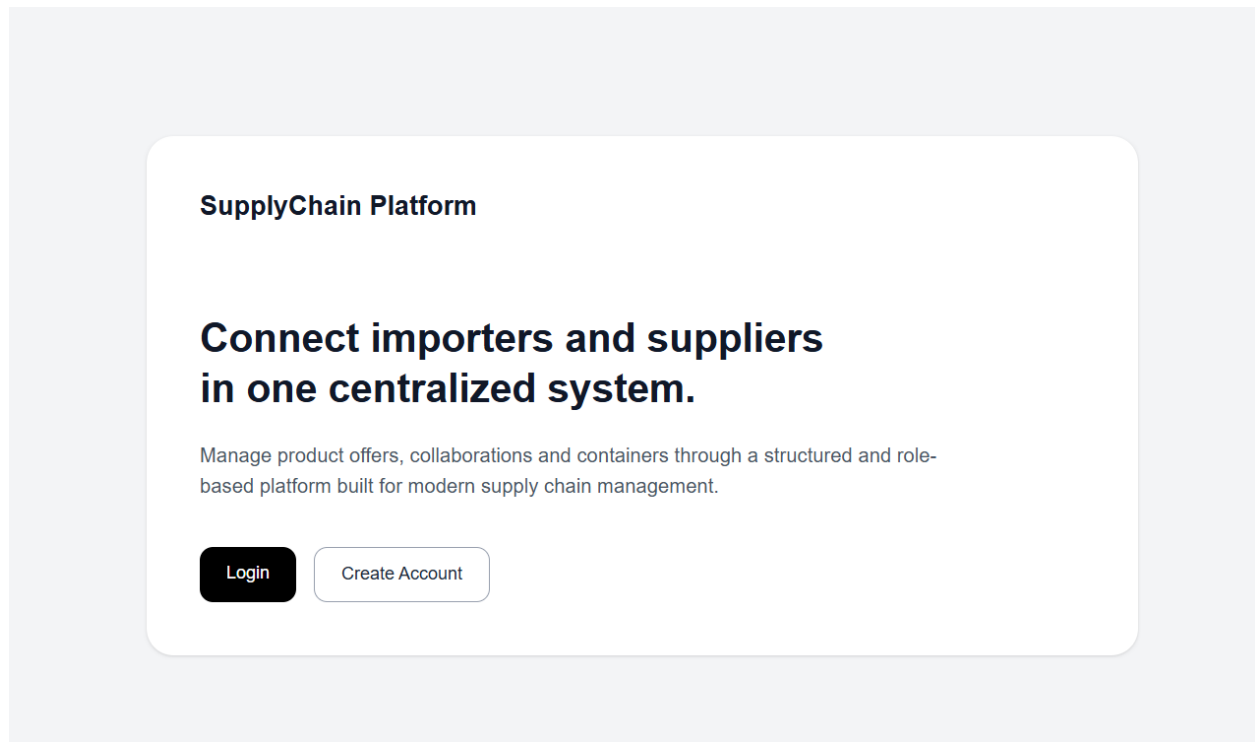
if (result.rowCount === 0) {
  return Response.json({ error: "Item not found" }, { status: 404 });
}

return Response.json({ success: true });

```

Provrava se da li kontejner postoji, kao i da li je njegov status "DRAFT". Ako je sve kako treba poziva se sql upit koji brise trazeni proizvod. Ukoliko taj proizvod ne postoji vraca se greska.

4.2. Prikaz korisničkog interfejsa (Frontend UI)



- Naziv ekrana: Landing Page
- Prikazuje naziv i kratki opis sistema za povezivanje uvoznika i dobavljača. Dostupna su dva dugmeta — "Login" za prijavu postojećih korisnika i "Create Account" za registraciju novih korisnika.

Login

Log in to access the app.

Email

pera@gmail.com

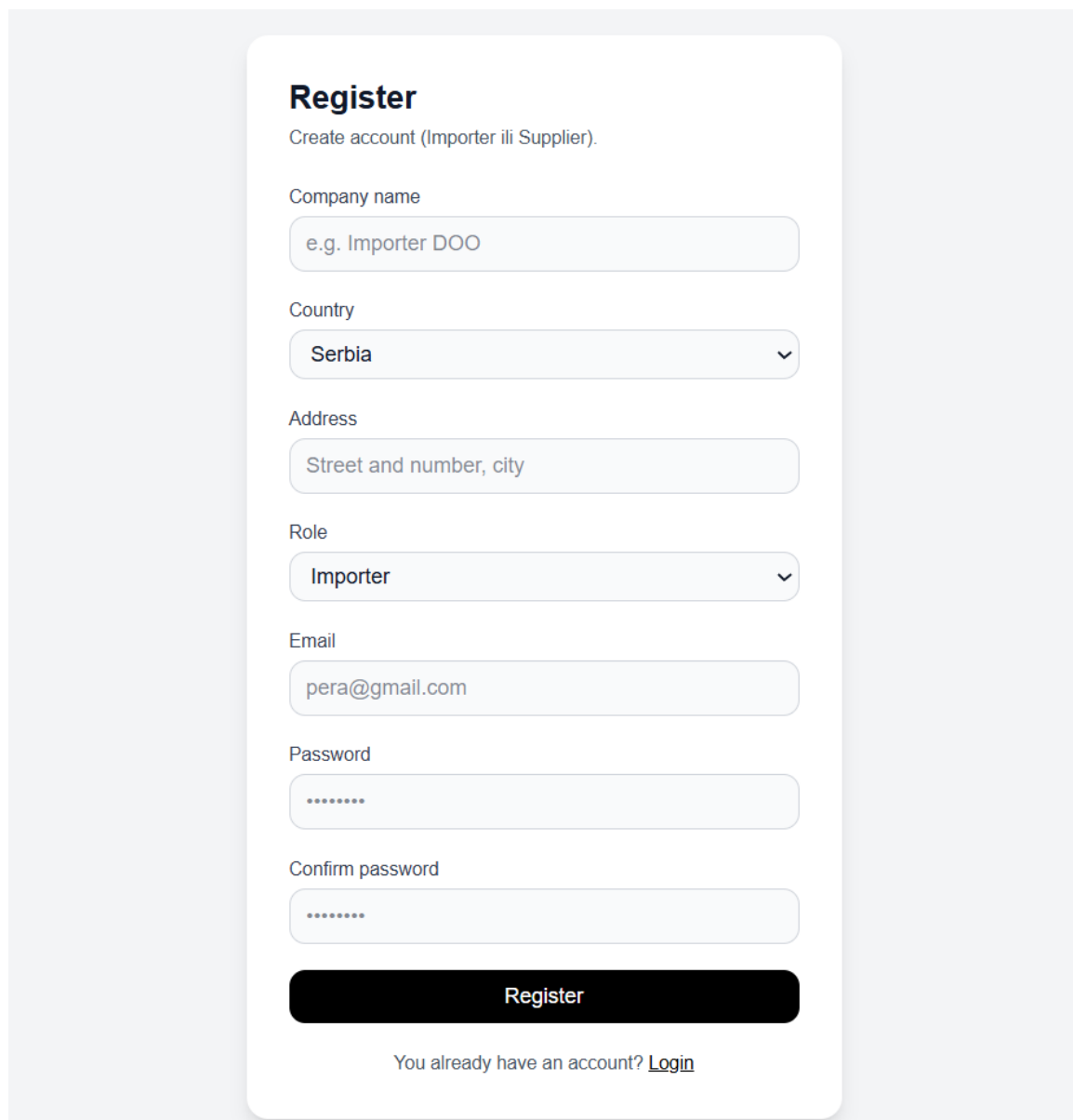
Password

.....

Login

You don't have an account? [Register](#)

- Naziv ekrana: Login
- Na ekranu se nalaze polja za unos imejl adrese i lozinke korisnika. Prikazano je dugme za prijavu putem kojeg korisnik potvrđuje unete podatke i pokreće proces autentifikacije. Takođe, dostupno je i dugme za registraciju koje omogućava prelazak na formu za kreiranje novog korisničkog naloga. Forma je jednostavna i omogućava brz pristup sistemu.

A registration form titled "Register" with a subtitle "Create account (Importer ili Supplier)". The form contains several input fields: "Company name" with a placeholder "e.g. Importer DOO", "Country" with a dropdown menu showing "Serbia", "Address" with a placeholder "Street and number, city", "Role" with a dropdown menu showing "Importer", "Email" with a placeholder "pera@gmail.com", "Password" with a masked input ".....", and "Confirm password" with a masked input ".....". At the bottom, there is a black "Register" button and a link "You already have an account? [Login](#)".

Register

Create account (Importer ili Supplier).

Company name

Country

Address

Role

Email

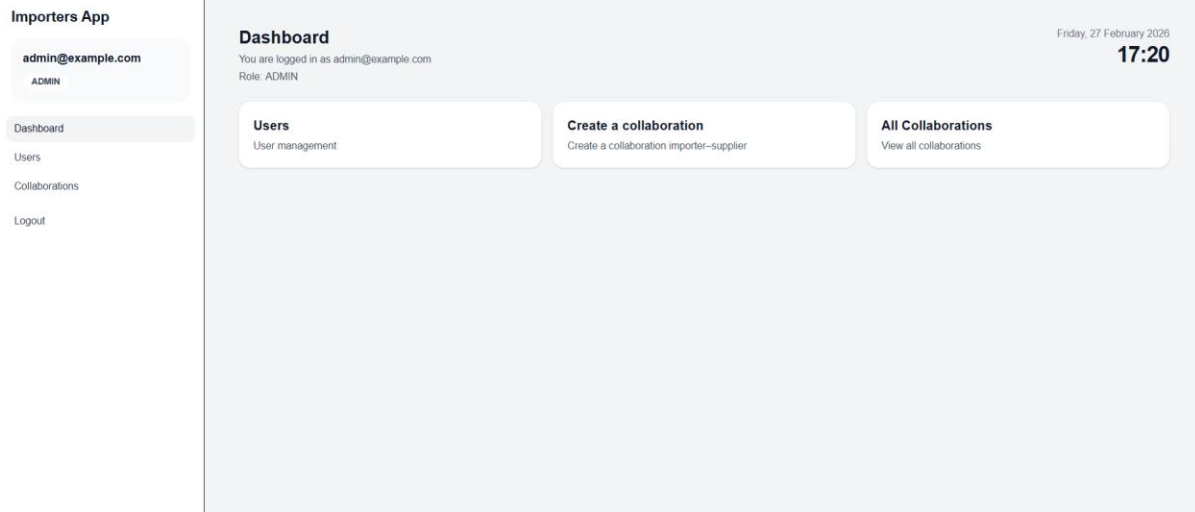
Password

Confirm password

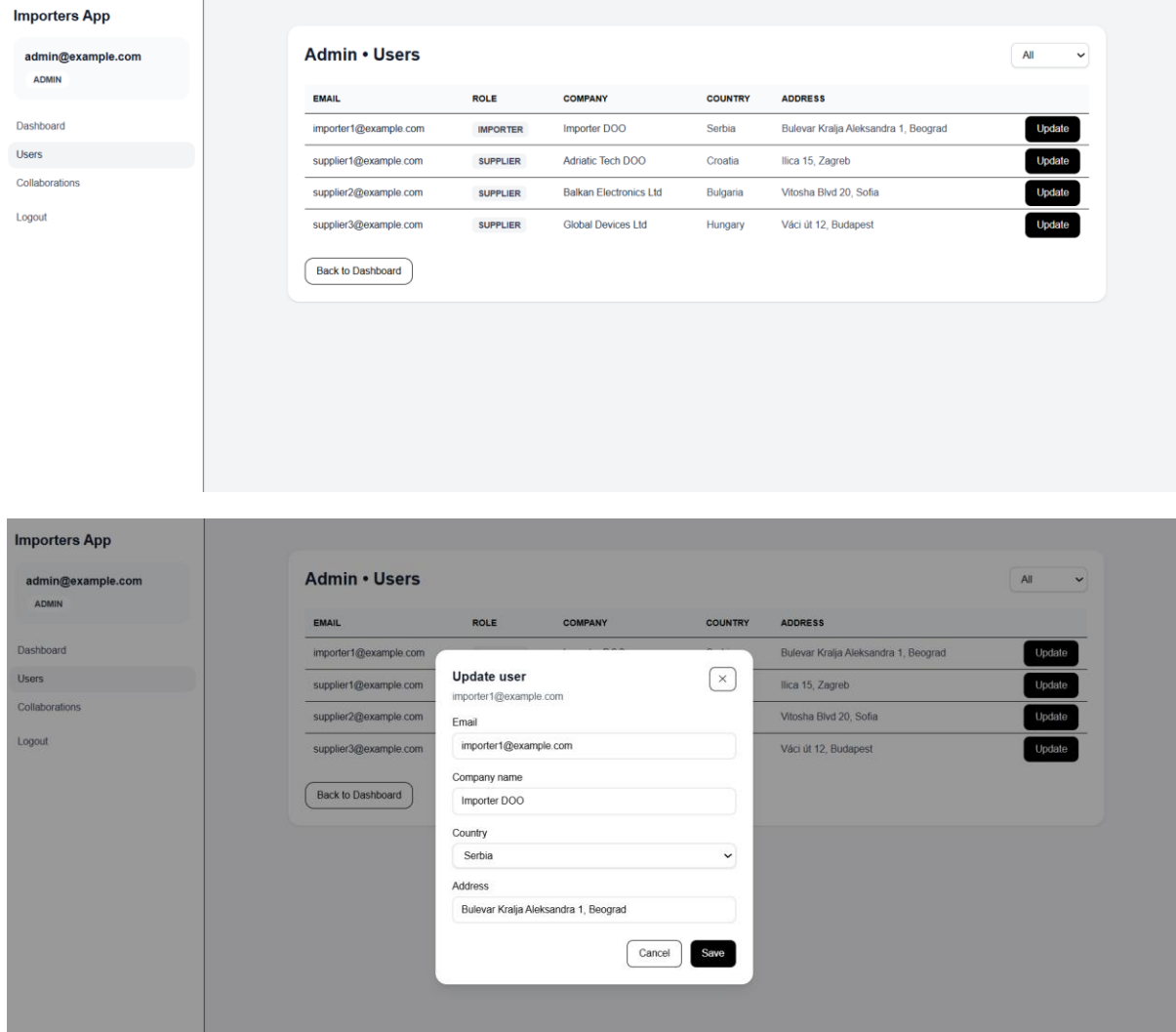
Register

You already have an account? [Login](#)

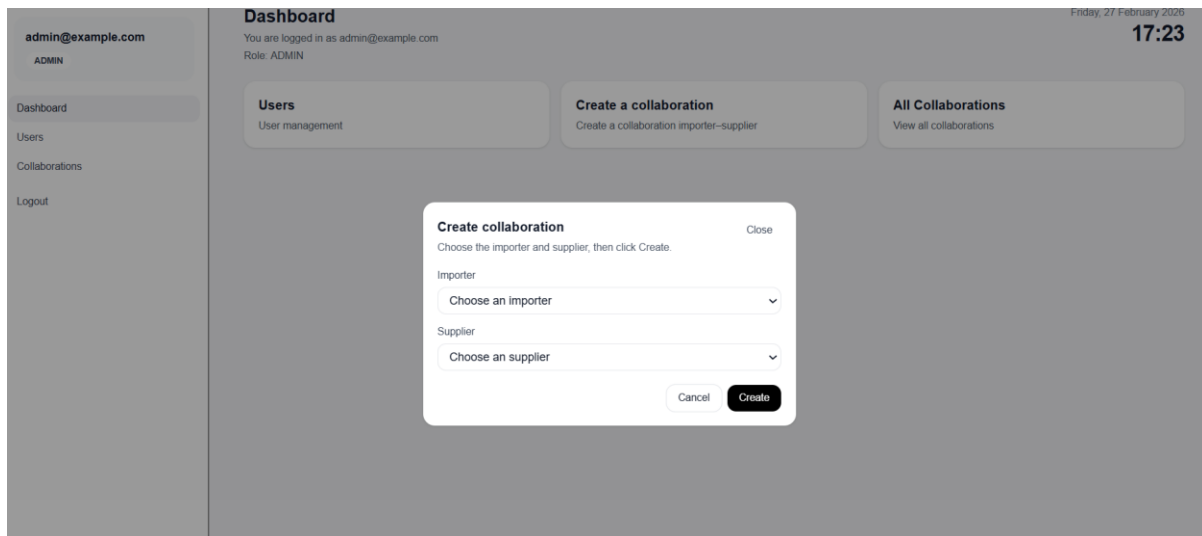
- Naziv ekrana: Register
- Ekran za registraciju omogućava kreiranje novog korisničkog naloga u sistemu. Korisniku su dostupna polja za unos naziva kompanije, države, adrese, izbora uloge (uvoznik ili dobavljač), imejl adrese, lozinke i potvrde lozinke. Na dnu forme nalazi se dugme za registraciju kojim se potvrđuje unos podataka. Takođe, prikazan je i link za povratak na ekran za prijavu za korisnike koji već imaju nalog



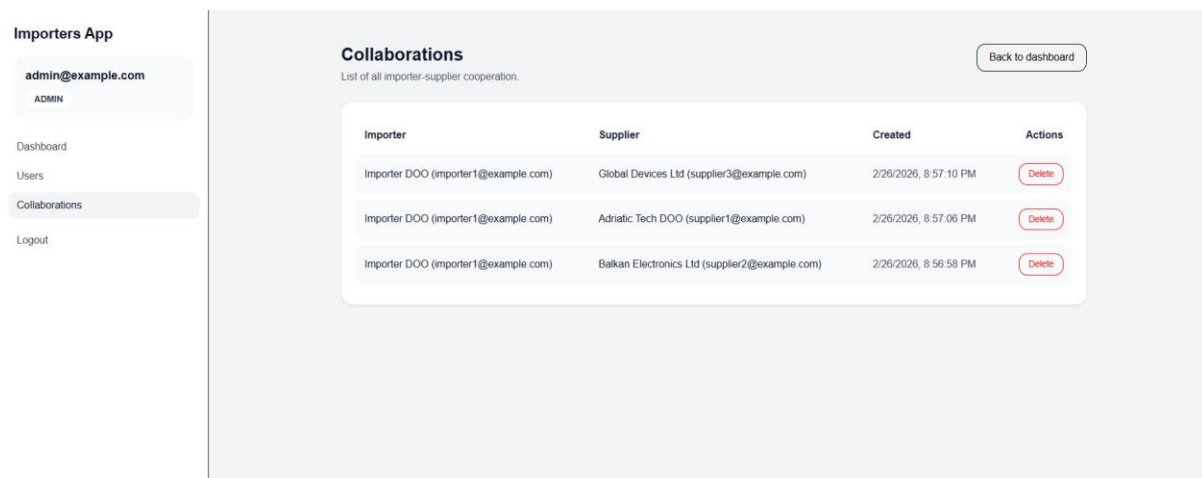
- Naziv ekrana: Dashboard (Admin)
- Glavni ekran nakon prijave za korisnika sa ADMIN ulogom. Prikazuje informacije o ulogovanom korisniku i trenutno vreme koje se ažurira u realnom vremenu. Sa leve strane nalazi se navigaciona traka sa opcijama: Dashboard, Users, Collaborations i Logout. Na glavnom delu ekrana prikazane su tri kartice — "Users" za upravljanje korisnicima, "Create a collaboration" za kreiranje nove saradnje između uvoznika i dobavljača, i "All Collaborations" za pregled svih postojećih saradnji.



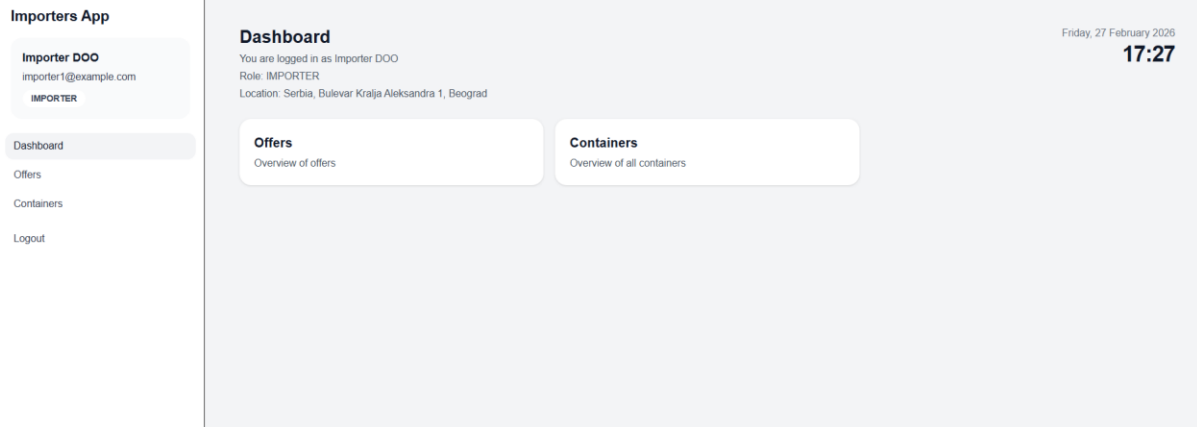
- Naziv ekrana: Upravljanje korisnicima
- Ekran prikazuje tabelarni pregled svih registrovanih korisnika u sistemu. Tabela sadrži sledeće kolone: email adresu korisnika, ulogu (IMPORTER ili SUPPLIER - prikazane kao oznake sa sivom pozadinom), naziv kompanije, zemlju i adresu. Za svakog korisnika u poslednjoj koloni nalazi se crno dugme "Update" za ažuriranje podataka.
- Nakon klika na dugme "Update" u tabeli korisnika, otvara se forma koja omogućava izmenu podataka izabranog korisnika.



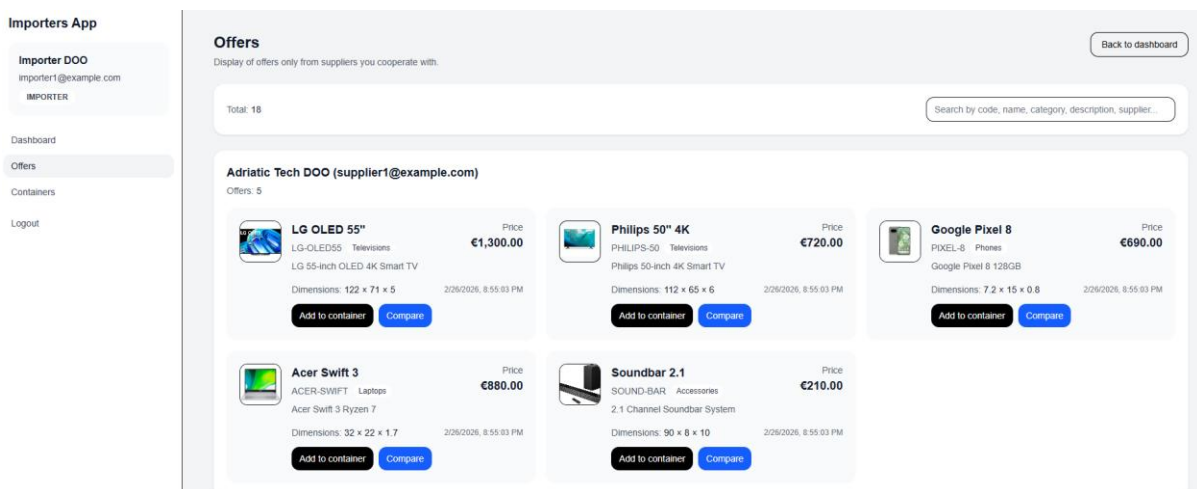
- Naziv ekrana: Kreiranje saradnje
- Nakon klika na opciju za kreiranje saradnje, otvara se forma koja omogućava administratoru da poveže uvoznika i dobavljača u novu saradnju.



- Naziv ekrana: Pregled saradnji
- Ekran prikazuje listu svih saradnji između uvoznika i dobavljača. Tabela sadrži kolone: uvoznik, dobavljač, datum kreiranja i akcije. Za svaku saradnju dostupno je dugme "Delete" za brisanje. U gornjem desnom uglu nalazi se dugme "Back to Dashboard" za povratak na glavni ekran.



- Naziv ekrana: Dashboard (Importer)
- Glavni ekran za korisnika sa IMPORTER ulogom. Prikazuje informacije o ulogovanom korisniku — naziv kompanije, email, ulogu i lokaciju, kao i trenutno vreme koje se ažurira u realnom vremenu. Na ekranu su dostupne dve kartice — "Offers" za pregled ponuda dobavljača i "Containers" za pregled svih kontejnera. Sa leve strane nalazi se navigaciona traka sa opcijama: Dashboard, Offers, Containers i Logout.



- Naziv ekrana: Offers (Importer)
- Ekran prikazuje ponude isključivo od dobavljača sa kojima uvoznik sarađuje. Na vrhu je prikazan ukupan broj ponuda i polje za pretragu po kodu, nazivu, kategoriji, opisu ili dobavljaču. Ponude su grupisane po dobavljačima, a svaka kartica proizvoda sadrži sliku, naziv, šifru, kategoriju, opis, dimenzije, datum i cenu. Za svaki proizvod dostupna su dugmad "Add to container" za dodavanje u kontejner i "Compare" za poređenje sa drugim proizvodima.

Importers App

Importer DOO
importer1@example.com
IMPORTER

Dashboard

Offers

Containers

Logout

Product Comparison

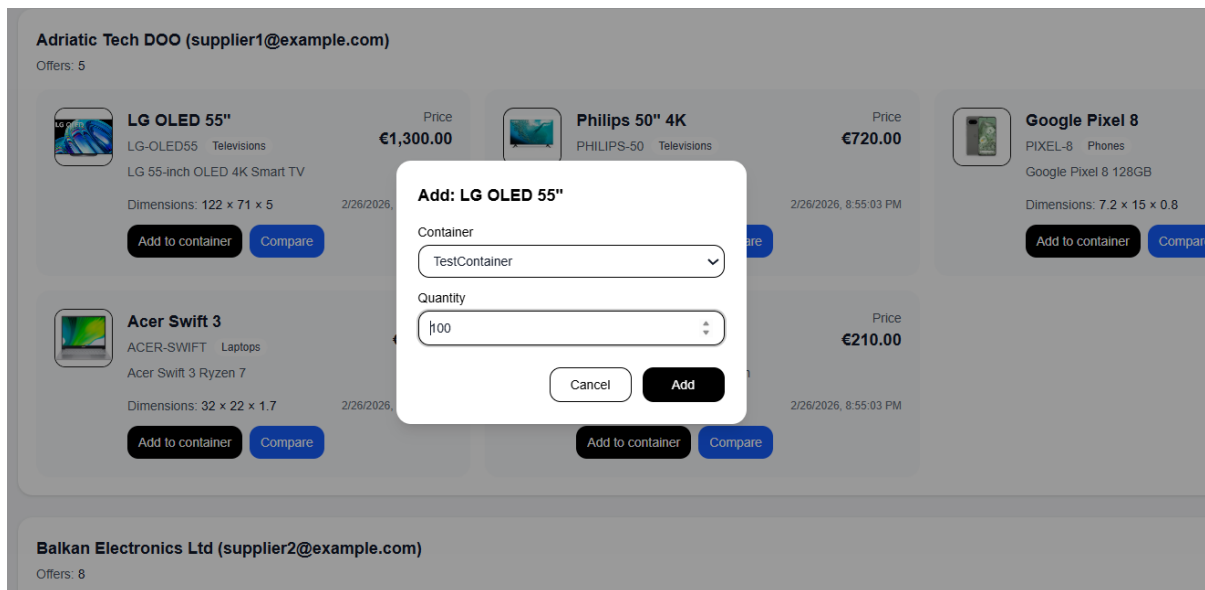
EUR

Back to offers

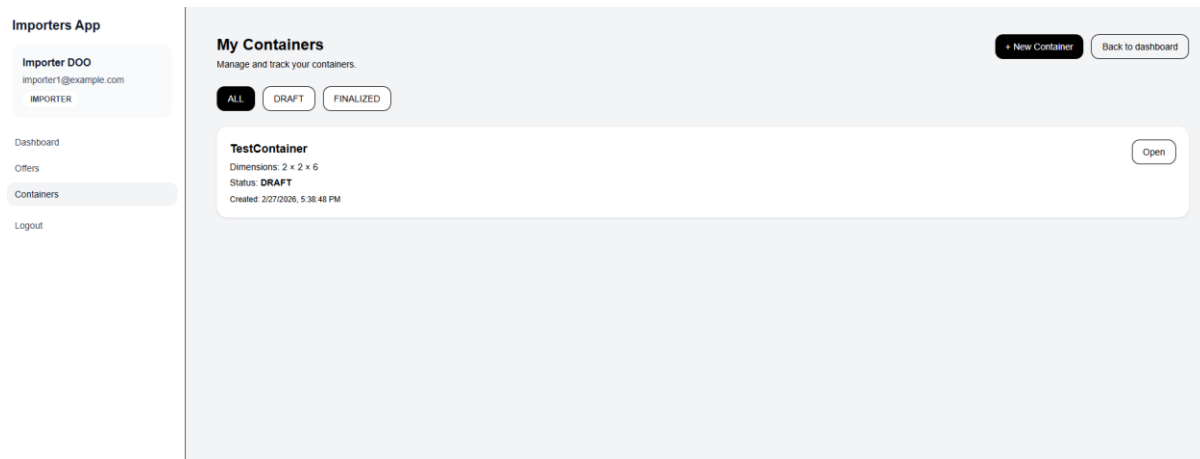
Feature	LG OLED 55"	TCL 65" 4K
Image		
Code	LG-OLED55	TCL-65
Category	Televisions	Televisions
Price	€1,300.00	€950.00
Width	122	145
Height	71	83
Depth	5	6
Description	LG 55-inch OLED 4K Smart TV	TCL 65-inch 4K UHD Smart TV
Created At	2/26/2026, 8:55:03 PM	2/26/2026, 8:55:03 PM

EUR
RSD
BAM
MKD
HUF
BGN
RON

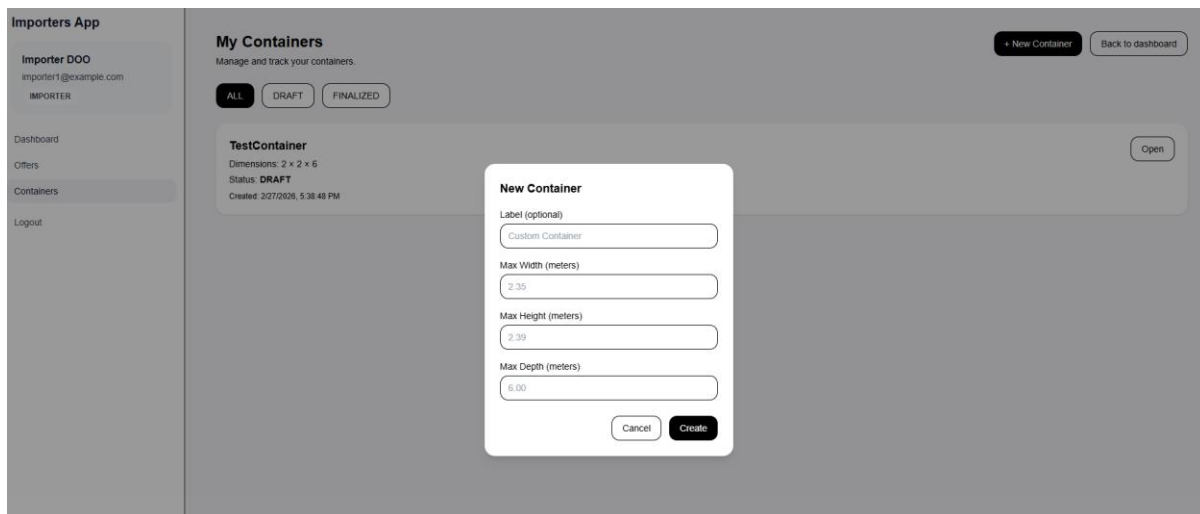
- Naziv ekrana: Product Comparison
- Ekran omogućava poređenje dva proizvoda jedan pored drugog. Tabela prikazuje karakteristike oba proizvoda — sliku, šifru, kategoriju, cenu, širinu, visinu, dubinu, opis i datum kreiranja. Bolja vrednost u poređenju je istaknuta zelenom bojom. U gornjem desnom uglu dostupan je padajući meni za izbor valute u kojoj se prikazuju cene. Dugme "Back to offers" vraća korisnika na ekran sa ponudama.



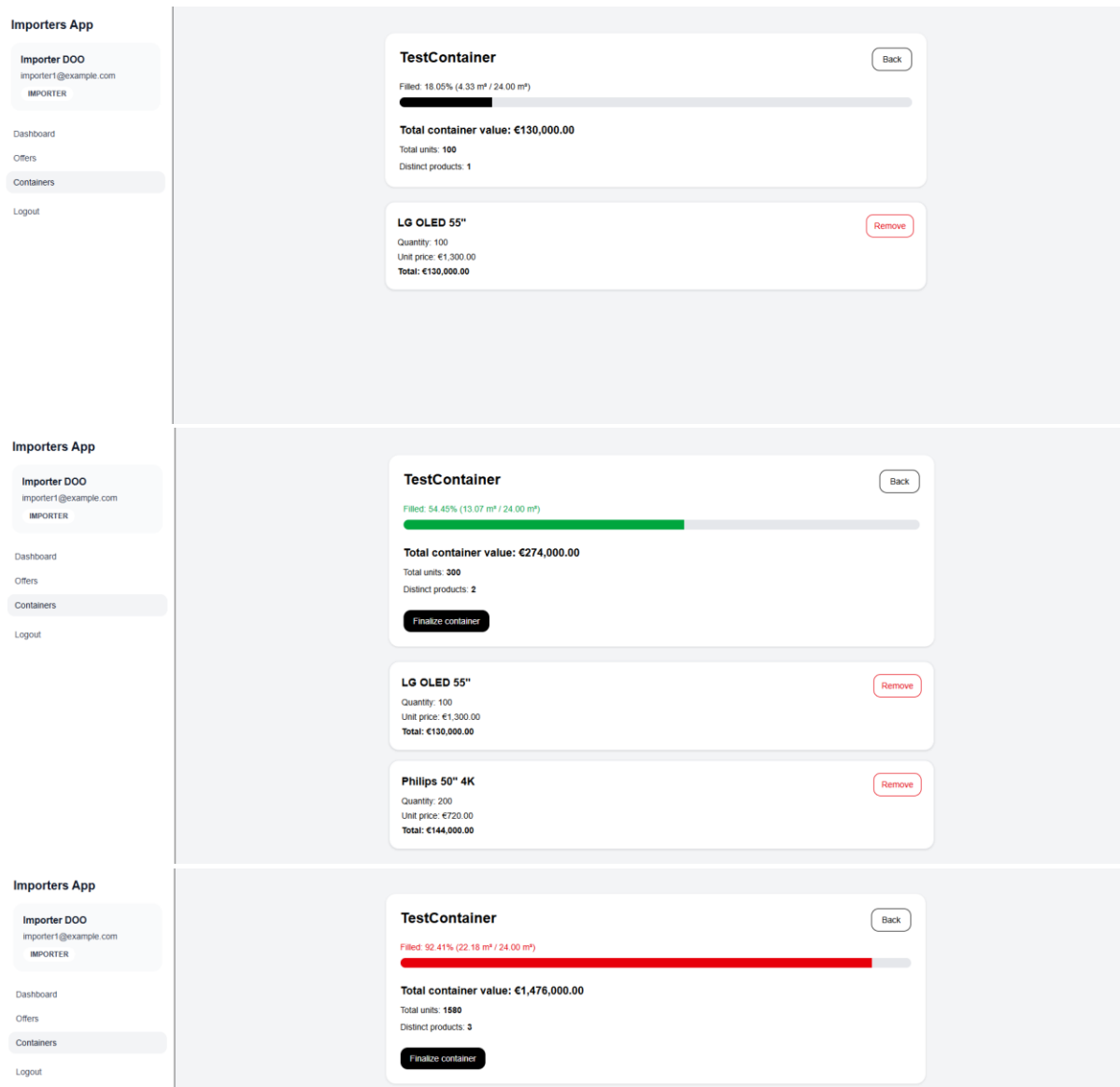
- Naziv ekrana: Add to Container
- Modalni dijalog koji se otvara klikom na dugme "Add to container" na kartici proizvoda. Prikazuje naziv proizvoda koji se dodaje, padajući meni za izbor kontejnera u koji se proizvod dodaje i polje za unos količine. Dostupna su dugmad "Cancel" za zatvaranje bez čuvanja i "Add" za potvrdu dodavanja proizvoda u izabrani kontejner.



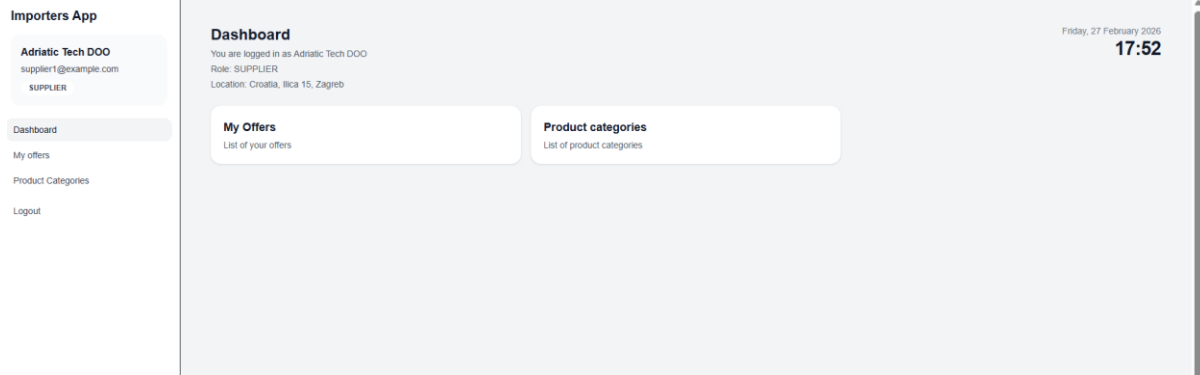
- Naziv ekrana: My Containers
- Ekran za upravljanje i praćenje kontejnera uvoznika. Dostupna su tri filtera — "All", "Draft" i "Finalized" za filtriranje kontejnera po statusu. U gornjem desnom uglu nalazi se dugme "+ New Container" za kreiranje novog kontejnera i dugme "Back to Dashboard" za povratak.



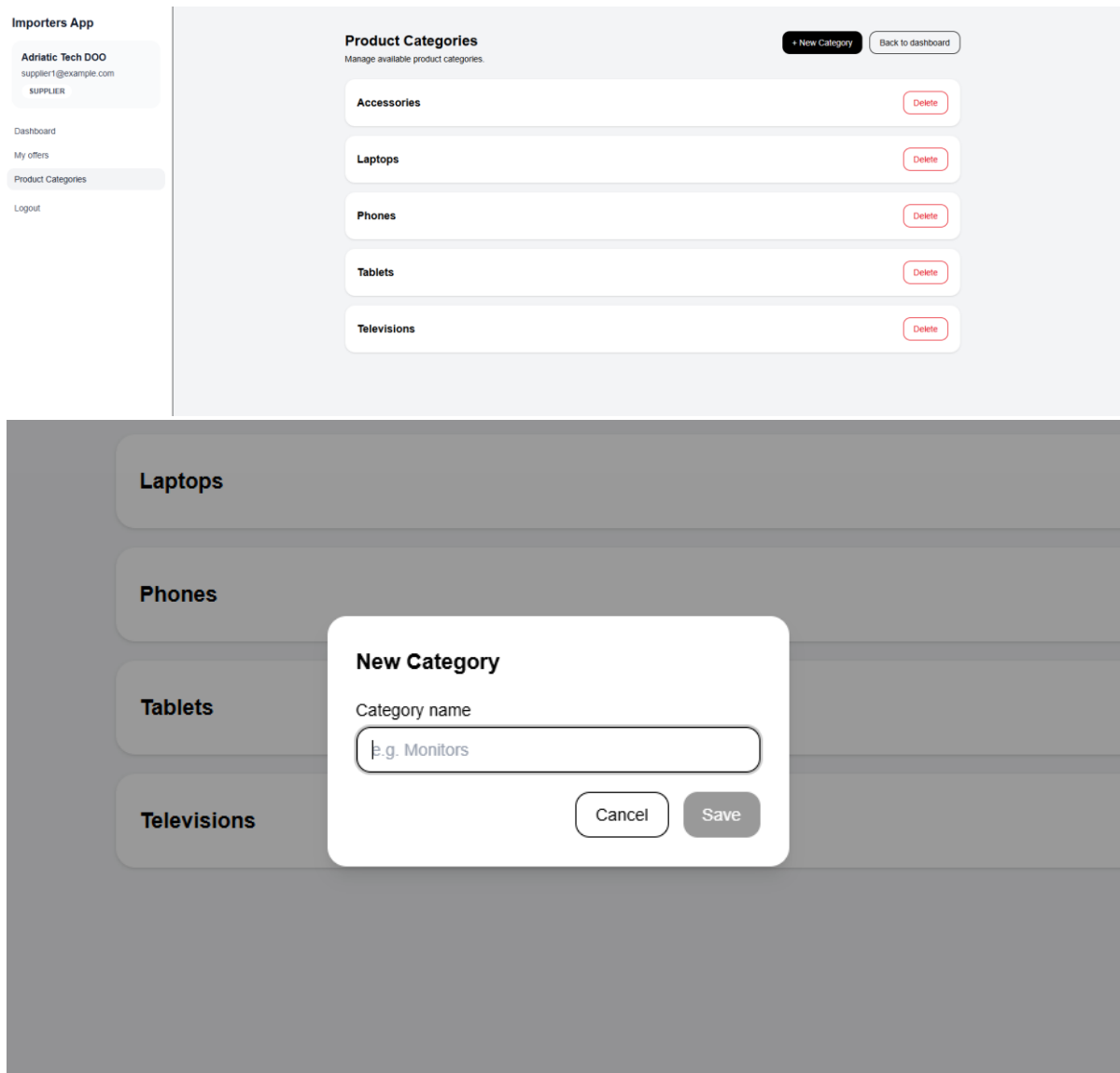
- Naziv ekrana: New Container
- Korisnik unosi opcionalnu oznaku kontejnera i maksimalne dimenzije — širinu, visinu i dubinu u metrima. Dostupna su dugmad "Cancel" za zatvaranje dijaloga bez čuvanja i "Create" za kreiranje novog kontejnera.



- Naziv ekrana: Container Details
- Ekran prikazuje detalje izabranog kontejnera. Na vrhu je prikazan naziv kontejnera, procenat popunjenosti i progress bar koji vizuelno prikazuje popunjenost — crna boja kada je kontejner malo popunjen, zelena kada je srednje popunjen i crvena kada je blizu maksimalnog kapaciteta. Prikazani su ukupna vrednost kontejnera, ukupan broj jedinica i broj različitih proizvoda.
- Ispod statistika nalaze se kartice svakog proizvoda u kontejneru sa nazivom, količinom, jediničnom cenom i ukupnom cenom. Za svaki proizvod dostupno je dugme "Remove" za uklanjanje iz kontejnera. Dugme "Finalize container" dostupno je isključivo kada je progress bar zelene boje ili crvene boje, odnosno kada popunjenost kontejnera nije prekoračila dozvoljen kapacitet. Dugme "Back" vraća korisnika na listu kontejnera.



- Naziv ekrana: Dashboard (Supplier)
- Glavni ekran za korisnika sa SUPPLIER ulogom. Prikazuje informacije o ulogovanom korisniku — naziv kompanije, email, ulogu i lokaciju, kao i trenutno vreme koje se ažurira u realnom vremenu. Na ekranu su dostupne dve kartice — "My Offers" za pregled sopstvenih ponuda i "Product Categories" za pregled kategorija proizvoda. Sa leve strane nalazi se navigaciona traka sa opcijama: Dashboard, My Offers, Product Categories i Logout.



- Naziv ekrana: Product Categories
- Ekran prikazuje listu svih dostupnih kategorija proizvoda. Svaka kategorija prikazana je kao kartica sa nazivom i dugmetom "Delete" za brisanje. U gornjem desnom uglu nalazi se dugme "+ New Category" koje otvara modalni dijalog za kreiranje nove kategorije, gde korisnik unosi naziv i potvrđuje klikom na "Save" ili odustaje klikom na "Cancel". Dugme "Back to Dashboard" vraća korisnika na glavni ekran.

Importers App

Adriatic Tech DOO

supplier1@example.com

SUPPLIER

Dashboard

My offers

Product Categories

Logout

My Offers

List of your bids published as a supplier.

+ New Offer

Back to dashboard

Total: 5

Search by code, name, category, description...



LG OLED55

LG 55-inch OLED 4K Smart TV

Dimensions: 122 x 71 x 5

Price

€1,300.00

2/26/2026, 8:55:03 PM

Delete



Philips 50" 4K

PHILIPS-50 Televisions

Philips 50-inch 4K Smart TV

Dimensions: 112 x 65 x 6

Price

€720.00

2/26/2026, 8:55:03 PM

Delete



Google Pixel 8

PIXEL-8 Phones

Google Pixel 8 128GB

Dimensions: 7.2 x 15 x 0.8

Price

€690.00

2/26/2026, 8:55:03 PM

Delete



Acer Swift 3

ACER-SWIFT Laptops

Acer Swift 3 Ryzen 7

Dimensions: 32 x 22 x 1.7

Price

€880.00

2/26/2026, 8:55:03 PM

Delete



Soundbar 2.1

SOUND-BAR Accessories

2.1 Channel Soundbar System

Dimensions: 90 x 8 x 10

Price

€210.00

2/26/2026, 8:55:03 PM

Delete

Importers App

Adriatic Tech DOO

supplier1@example.com

SUPPLIER

Dashboard

My offers

Product Categories

Logout

My Offers

List of your bids published as a supplier.

+ New Offer

Back to dashboard

Total: 5 | Shown: 1

laptops



Acer Swift 3

ACER-SWIFT Laptops

Acer Swift 3 Ryzen 7

Dimensions: 32 x 22 x 1.7

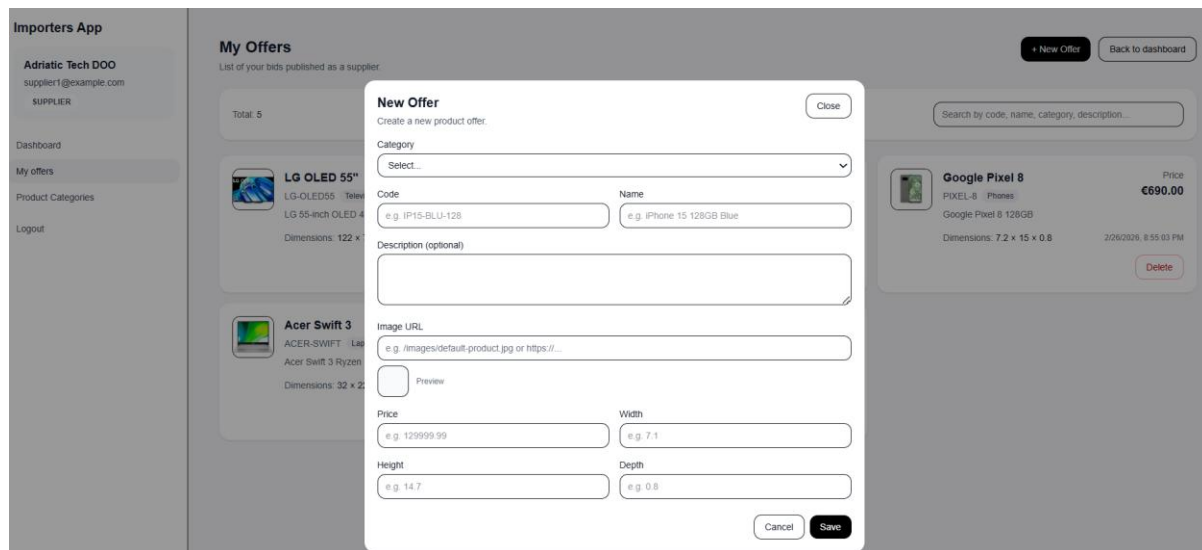
Price

€880.00

2/26/2026, 8:55:03 PM

Delete

- Naziv ekrana: My Offers (Supplier)
- Ekran prikazuje listu svih ponuda koje je dobavljač objavio. Na vrhu je prikazan ukupan broj ponuda i polje za pretragu po kodu, nazivu, kategoriji ili opisu. Svaka kartica proizvoda sadrži sliku, naziv, šifru, kategoriju, opis, dimenzije, datum i cenu. Za svaku ponudu dostupno je dugme "Delete" za brisanje. U gornjem desnom uglu nalazi se dugme "+ New Offer" za dodavanje nove ponude i dugme "Back to Dashboard" za povratak.



- Naziv ekrana: New Offer
- Modalni dijalog koji se otvara klikom na dugme "+ New Offer". Dobavljač unosi sve potrebne podatke o novoj ponudi — kategoriju putem padajućeg menija, šifru, naziv, opcioni opis, URL slike sa mogućnošću pretpregleda, cenu i dimenzije proizvoda (širina, visina, dubina). Dostupna su dugmad "Cancel" za odustajanje i "Save" za čuvanje nove ponude, kao i dugme "Close" u gornjem desnom uglu modalnog prozora.

4.3. Povezanost zahteva i implementacije

Zahtev	Modul	API ruta	UI ekran
Prijava korisnika	Auth	POST /api/auth/login	LoginPage
Registracija korisnika	Auth	POST /api/auth/register	RegisterPage
Odjava korisnika	Auth	POST /api/auth/logout	Dashboard
Pregled trenutnog korisnika	Auth	GET /api/auth/me	AppShell
Pregled korisnika	AdminUsers	GET /api/admin/users	AdminUsersPage
Izmena korisnika	AdminUsers	PUT /api/admin/users/{id}	UpdateUserModal
Pregled saradnji	Collaborations	GET /api/admin/collaborations	CollaborationsPage
Kreiranje saradnje	Collaborations	POST /api/admin/collaborations	CreateCollaborationModal
Brisanje saradnje	Collaborations	DELETE /api/admin/collaborations/{id}	CollaborationsPage
Pregled kategorija proizvoda	Categories	GET /api/categories	ProductCategoriesPage
Dodavanje kategorije	Categories	POST /api/categories	NewCategoryModal
Brisanje kategorije	Categories	DELETE /api/categories/{id}	ProductCategoriesPage
Pregled kontejnera	Containers	GET /api/importer/containers	ImporterContainersPage
Kreiranje kontejnera	Containers	POST /api/importer/containers	NewContainerModal
Pregled kontejnera	Containers	GET /api/importer/containers/{id}	ContainerDetailsPage
Finalizacija kontejnera	Containers	POST /api/importer/containers/{id}/finalize	ContainerDetailsPage
Brisanje stavke iz kontejnera	ContainerItems	DELETE /api/importer/containers/{id}/items/{itemId}	ContainerDetailsPage

Zahtev	Modul	API ruta	UI ekran
Dodavanje stavke u kontejner	ContainerItems	POST /api/containerItems	AddToContainerModal
Pregled ponuda dobavljača	SupplierOffers	GET /api/supplier/offers	SupplierOffersPage
Kreiranje ponude	SupplierOffers	POST /api/supplier/offers	NewOfferModal
Brisanje ponude	SupplierOffers	DELETE /api/supplier/offers/{id}	SupplierOffersPage
Pregled ponuda za uvoznika	ImporterOffers	GET /api/importer/offers	ImporterOffersPage
Poređenje ponuda	ImporterOffers	GET /api/importer/offers/compare	ComparePage
Kurs valuta	Exchange	GET /api/exchange	Dashboard
Server vreme	Time	GET /api/time	Dashboard
API dokumentacija	Swagger	GET /api/swagger	ApiDocsPage

5. Tehnička realizacija i DevOps postavka projekta

5.1. Dockerizacija aplikacije

Aplikacija je dockerizovana korišćenjem **Docker-a i docker-compose alata**, čime je omogućeno jednostavno pokretanje kompletnog sistema u izolovanom okruženju. Docker omogućava da se aplikacija pokreće zajedno sa svim zavisnostima bez potrebe za ručnim instaliranjem softvera na host sistemu.

Struktura Dockerfile fajla

Dockerfile definiše način na koji se gradi Docker image za Next.js aplikaciju.

- Kao osnovna slika koristi se **node:20-alpine**, koja predstavlja laganu verziju Node.js okruženja zasnovanu na Alpine Linux distribuciji.
- Direktiva **WORKDIR /app** postavlja radni direktorijum unutar kontejnera u kome će se nalaziti aplikacija.
- Komandom **COPY package*.json ./** kopiraju se fajlovi **package.json** i **package-lock.json**, koji sadrže definisane zavisnosti projekta.
- Komandom **RUN npm install** instaliraju se sve potrebne Node.js zavisnosti.
- Komandom **COPY . .** kopira se ostatak aplikacije u kontejner.
- Direktiva **EXPOSE 3000** označava da aplikacija koristi port **3000** za pristup web servisu.
- Na kraju se koristi komanda **CMD ["npm", "run", "dev"]** kojom se pokreće razvojni server Next.js aplikacije unutar kontejnera.

Korišćenje docker-compose.yml

Za pokretanje više servisa koristi se **compose.yaml** fajl koji definiše konfiguraciju kontejnera i njihov međusobni odnos.

U okviru docker-compose konfiguracije definisana su dva servisa:

app servis

Ovaj servis predstavlja Next.js aplikaciju (frontend i backend deo aplikacije).

Konfiguracija obuhvata:

- build: . – Docker image se gradi na osnovu lokalnog Dockerfile fajla
- ports: "3000:3000" – mapiranje porta 3000 iz kontejnera na port 3000 host sistema
- volumes – omogućava deljenje fajlova između host sistema i kontejnera:
 - ./app omogućava da se promene u kodu odmah reflektuju u kontejneru
 - /app/node_modules sprečava konflikt lokalnih i kontejnerskih zavisnosti
- depends_on: db – definiše da aplikacija zavisi od baze podataka
- env_file: .env – učitava promenljive okruženja iz .env fajla

db servis

Ovaj servis predstavlja **PostgreSQL bazu podataka**.

Konfiguracija uključuje:

- korišćenje zvanične slike postgres:17
- automatski restart kontejnera pomoću restart: unless-stopped
- promenljive okruženja za inicijalizaciju baze:
 - POSTGRES_USER
 - POSTGRES_PASSWORD
 - POSTGRES_DB
- mapiranje porta 5432:5432
- definisanje **Docker volumena** importers_pgdata za trajno čuvanje podataka baze

Na ovaj način podaci baze ostaju sačuvani čak i kada se kontejner restartuje.

Pokretanje servisa (frontend, backend, baza)

U okviru aplikacije:

- **Frontend i backend** su objedinjeni u jednoj **Next.js aplikaciji (app servis)**.
- **Baza podataka** se pokreće u posebnom **PostgreSQL kontejneru (db servis)**.

Servis aplikacije komunicira sa bazom preko Docker mreže koju docker-compose automatski kreira.

.dockerignore fajl

U projektu se koristi i **.dockerignore** fajl kako bi se smanjila veličina Docker image-a i ubrzalo njegovo građenje.

Ignorisani su sledeći fajlovi i direktorijumi:

- node_modules
- .next
- .git
- .env
- npm-debug.log
- Dockerfile
- docker-compose.yml

Na ovaj način se izbegava kopiranje nepotrebnih fajlova u Docker image.

Komande za lokalno pokretanje projekta pomoću Dockera

Za pokretanje aplikacije koriste se sledeće komande:

Pokretanje svih servisa i build image-a:

- `docker compose up --build`

Pokretanje servisa u pozadini:

- `docker compose up -d`

Prikaz aktivnih kontejnera:

- `docker compose ps`

Praćenje logova aplikacije:

- `docker compose logs -f app`

Zaustavljanje i uklanjanje kontejnera:

- `docker compose down`

5.2. API specifikacija (Swagger)

Za dokumentovanje REST API-ja u aplikaciji korišćen je **Swagger (OpenAPI)** alat. Swagger omogućava automatsko generisanje dokumentacije svih API endpoint-a, kao i njihovo testiranje putem interaktivnog web interfejsa.

U aplikaciji se Swagger specifikacija generiše pomoću biblioteke **swagger-jsdoc**, koja na osnovu Swagger komentara unutar API ruta automatski kreira **OpenAPI 3.0 specifikaciju**. Generisana specifikacija dostupna je preko rute:

- **<http://localhost:3000/api/swagger>**

Za prikaz dokumentacije koristi se **Swagger UI**, implementiran pomoću biblioteke **swagger-ui-react**. Ova komponenta učitava generisanu specifikaciju i prikazuje je u preglednom interfejsu.

Swagger dokumentaciji se može pristupiti otvaranjem sledeće stranice u browseru:

- **<http://localhost:3000/api-docs>**

Na toj stranici prikazuje se lista svih dostupnih API endpoint-a, gde je moguće videti njihove opise, parametre, strukturu zahteva i odgovora, kao i direktno testirati pozive API-ja.

Svaka API ruta u projektu dokumentovana je pomoću Swagger komentara koji definišu:

- HTTP metodu i putanju endpoint-a
- opis funkcionalnosti
- strukturu zahteva (request body)
- moguće odgovore servera i status kodove

Na ovaj način API dokumentacija se automatski generiše i ažurira zajedno sa promenama u kodu aplikacije, što značajno olakšava razvoj i testiranje sistema.

5.3. Korišćeni eksterni API-ji

U okviru aplikacije korišćeni su eksterni API servisi kako bi se dobili dodatni podaci koji nisu deo lokalne baze sistema. Komunikacija sa eksternim servisima vrši se putem **REST protokola**, a podaci se razmenjuju u **JSON formatu**.

1. TimeAPI – servis za trenutno vreme i datum

Naziv servisa: TimeAPI (timeapi.io)

Svrha u aplikaciji:

Ovaj API se koristi za dobijanje tačnog trenutnog vremena za vremensku zonu **Europe/Belgrade**. Dobijeno vreme se prikazuje na **Dashboard stranici aplikacije**, gde korisnik može videti trenutni datum i vreme.

Backend ruta /api/time poziva eksterni servis i vraća vreme dobijeno od API-ja zajedno sa trenutnim vremenom servera. Na osnovu razlike između ta dva vremena frontend sinhronizuje prikaz vremena u korisničkom interfejsu.

Osnovni endpoint i način komunikacije:

GET <https://timeapi.io/api/Time/current/zone?timeZone=Europe/Belgrade>

Komunikacija se vrši putem **HTTP GET zahteva**, a odgovor je u **JSON formatu**.

Primer zahteva:

GET <https://timeapi.io/api/Time/current/zone?timeZone=Europe/Belgrade>

Primer odgovora:

```
{
  "dateTime": "2026-03-09T21:35:10",
  "year": 2026,
  "month": 3,
  "day": 9,
  "hour": 21,
  "minute": 35,
  "seconds": 10
}
```

2. ExchangeRate API – servis za kurseve valuta

Naziv servisa: ExchangeRate API (exchangerate-api.com)

Svrha u aplikaciji:

Ovaj API se koristi za dobijanje aktuelnih kurseva valuta. U aplikaciji se koristi na stranici za **poređenje proizvoda**, gde korisnik može prikazati cenu proizvoda u različitim valutama.

Backend ruta /api/exchange poziva eksterni servis i vraća kursne liste sa baznom valutom **EUR**. Frontend zatim koristi te kurseve za konverziju cena proizvoda u izabranu valutu.

Osnovni endpoint i način komunikacije:

GET <https://api.exchangerate-api.com/v4/latest/EUR>

Komunikacija se vrši putem **HTTP GET zahteva**, a odgovor je u **JSON formatu**.

Primer zahteva:

GET <https://api.exchangerate-api.com/v4/latest/EUR>

Primer odgovora:

```
{
  "base": "EUR",
  "date": "2026-03-07",
  "rates": {
    "USD": 1.08,
    "RSD": 117.2,
    "BAM": 1.95,
    "MKD": 61.5
  }
}
```

Dobijeni kursevi se zatim koriste u aplikaciji za preračunavanje cena proizvoda u različite valute.

5.4. Dokumentacija projekta (README)

Opis aplikacije

Importers App je web aplikacija za upravljanje ponudama proizvoda i saradnjama između **uvoznika** i **dobavljača**. Sistem podržava tri tipa korisnika: **ADMIN**, **IMPORTER** i **SUPPLIER**.

Aplikacija omogućava registraciju i prijavu korisnika, upravljanje ponudama i kategorijama proizvoda, upravljanje kontejnerima, poređenje proizvoda i administraciju korisnika i saradnji.

API dokumentacija dostupna je putem **Swagger interfejsa** na adresi:

- Swagger UI: <http://localhost:3000/api-docs>
- OpenAPI JSON: <http://localhost:3000/api/swagger>

Korišćene tehnologije

- Next.js
- React
- TypeScript
- PostgreSQL
- Drizzle ORM
- Swagger / OpenAPI
- Docker i Docker Compose

Aplikacija takođe koristi eksterne API servise:

- TimeAPI – za dobijanje trenutnog vremena
- ExchangeRate API – za kursne liste valuta

Lokalno pokretanje aplikacije

1. Kloniraj repozitorijum i uđi u projekat:

```
git clone https://github.com/elab-development/internet-tehnologije-2025-aplikacijazauvoznike_2022_0252.git
cd internet-tehnologije-2025-aplikacijazauvoznike_2022_0252
```

2. Instaliraj zavisnosti:

```
npm install
```

3. Napravi .env fajl sa potrebnim promenljivama:

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/iteh
API_URL=http://localhost:3000
JWT_SECRET=arsa
JWT_EXPIRES=7d
```

4. Pokreni PostgreSQL bazu:

```
docker run --name importers-postgres
-e POSTGRES_USER=postgres
-e POSTGRES_PASSWORD=postgres
-e POSTGRES_DB=iteh
-p 5432:5432
-v importers_pgdata:/var/lib/postgresql/data
-d postgres:17
```

5. Pokreni migracije i inicijalno punjenje baze:

```
npm run db:migrate
npm run db:seed
```

6. Pokreni aplikaciju:

```
npm run dev
```

7. Otvori aplikaciju:

- **App:** <http://localhost:3000>
- **Swagger UI:** <http://localhost:3000/api-docs>
- **OpenAPI JSON:** <http://localhost:3000/api/swagger>

Pokretanje pomoću Docker-a i Docker Compose-a

1. Napravi .env fajl u root folderu projekta:

```
DATABASE_URL=postgres://postgres:postgres@db:5432/iteh
API_URL=http://localhost:3000
JWT_SECRET=arsa
JWT_EXPIRES=7d
```

2. Pokreni kontejnere:

```
docker compose up --build
```

3. Pokreni migracije i inicijalno punjenje baze:

```
docker compose exec app npm run db:migrate
docker compose exec app npm run db:seed
```

4. Otvori aplikaciju:

- **App:** <http://localhost:3000>
- **Swagger UI:** <http://localhost:3000/api-docs>

5. Zaustavljanje kontejnera:

```
docker compose down
```